some application capacity is impacted, and the application keeps on working. In concert with patterns in this book and in alignment with the services offered by the major cloud platforms, this approach is not only viable, but also attractive due to the great reduction in complexity and new economic efficiencies.

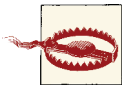
Hardware Failure Is Inevitable, but Not Frequent

Discussion of recovering from failures in commodity hardware can be misleading. Just because commodity hardware fails *more frequently* than high-end hardware does not mean it fails *frequently*. Hardware failures impact only a small percentage of the commodity servers in the data center every year. But be ready: eventually it will be your turn.

The cloud platform assumes that much of the MTTR duties are completed through automation, but also imposes requirements on your application, forming something of a partnership in handling failure.

Impact of Commodity Hardware on Application Logic

Cloud-native applications expect failure and are able to detect and automatically recover from common scenarios. Some of these failure scenarios are present because the application is relying on commodity hardware.



Commodity hardware fails occasionally. Plan on it happening to your compute nodes and plan on handling it.

Failure may simply be due to an issue with a specific physical server such as bad memory, a crashed disk drive, or coffee spilled on the motherboard. Other scenarios originate from software failures. For more information about responding to failures at the individual node level, refer to *Node Failure Pattern* (Chapter 10).

The failure scenario just described may be obvious: your application code is running on commodity hardware, and when that hardware fails your application is impacted. What is less obvious is that cloud services on which your application also depends (databases, persistent file storage, messaging, and so on) on are also running on commodity hardware. When these services experience a disruption due to a hardware failure, your application may also be impacted. In many scenarios, the cloud platform service recovers without any visible degradation, but sometimes capacity is temporarily reduced, forcing the calling application to handle transient failure. For more information about responding to failures encountered during calls to a cloud service, refer to Chapter 3.

Homogeneous Hardware

Cloud data centers also strive to use homogeneous hardware for easier management and maintenance of resources. Procurement of large scale homogeneous hardware is possible through inexpensive and readily available commodity hardware.

The level of homogeneity in the hardware is unlikely to directly impact applications as long as the allocated capacity in a virtual machine remains predictable.

Homogeneous Hardware Benefits in the Real World

Southwest Airlines is one of the most consistently profitable airlines in the world, in part fueled by their insistence on homogeneous commodity hardware: the Boeing 737. This is the only type of plane in the whole fleet, vastly reducing complexity in breadth of skills needed by mechanics and pilots, streamlining parts inventory, and probably even simplifying software that runs the airlines since there are fewer differences between flights.

Summary

Cloud platform vendors make choices around cost-efficiency that directly impact the architecture of applications. Architecting to deal with failure is part of what distinguishes a cloud-native application from a traditional application. Rather than attempting to shield the application from all failures, dealing with failure is a shared responsibility between the cloud platform and the application.

CHAPTER 9

Busy Signal Pattern

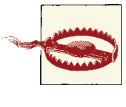
This pattern focuses on how an application should react when a cloud service responds to a programmatic request with a busy signal rather than success.

This pattern reflects the perspective of a client, not the service. The client is programmatically making a request of a service, but the service replies with a busy signal. The client is responsible for correct interpretation of the busy signal followed by an appropriate number of retry attempts. If the busy signals continue during retries, the client treats the service as unavailable.

Dialing a telephone occasionally results in a busy signal. The normal response is to retry, which usually results in a successful telephone call.

Similarly, invoking a service occasionally results in a failure code being returned, indicating the cloud service is not currently able to satisfy the request. The normal response is to retry which usually results in the service call succeeding.

The main reason a cloud service cannot satisfy a request is because it is too busy. Sometimes a service is "too busy" for just a few hundred milliseconds, or one or two seconds. Smart retry policies will help handle busy signals without compromising user experience or overwhelming busy services.



Applications that do not handle busy signals will be unreliable.

Context

The Busy Signal Pattern is effective in dealing with the following challenges:

- Your application uses cloud platform services that are not guaranteed to respond successfully every time

This pattern applies to accessing cloud platform services of all types, such as management services, data services, and more.

More generally, this pattern can be applied to applications accessing services or resources over a network, whether in the cloud or not. In all of these cases, periodic transient failures should be expected. A familiar non-cloud example is when a web browser fails to load a website fully, but a simple refresh or retry fixes the problem.

Cloud Significance

For reasons explained in *Multitenancy and Commodity Hardware Primer* (Chapter 8), applications using cloud services will experience periodic transient failures that result in a busy signal response. If applications do not respond appropriately to these busy signals, user experience will suffer and applications will experience errors that are difficult to diagnose or reproduce. Applications that expect and plan for busy signals can respond appropriately.

The pattern makes sense for robust applications even in on-premises environments, but historically has not been as important because such failures are far less frequent than in the cloud.

Impact

Availability, Scalability, User Experience

Mechanics

Use the Busy Signal Pattern to detect and handle normal transient failures that occur when your application (the *client* in this relationship) accesses a cloud service. A *transient failure* is a short-lived failure that is not the fault of the client. In fact, if the client reissues the identical request only milliseconds later, it will often succeed.

Transient failures are expected occurrences, not exceptional ones, similar to making a telephone call and getting a busy signal.

Busy Signals Are Normal

Consider making a phone call to a call center where your call will be answered by one of hundreds of agents standing by. Usually your call goes through without any problem, but not every time. Occasionally you get a busy signal. You don't suspect anything is wrong, you simply hit redial on your phone and usually you get through. This is a transient failure, with an appropriate response: retry.

However, many consecutive busy signals will be an indicator to stop calling for a while, perhaps until later in the day. Further, we only will retry if there is a true busy signal. If we've dialed the wrong number or a number that is no longer in service, we do not retry.

Although network connectivity issues might sometimes be the cause of transient failures, we will focus on transient failures at the service boundary, which is when a request reaches the cloud service, but is not immediately satisfied by the service. This pattern applies to any cloud service that can be accessed programmatically, such as relational databases, NoSQL databases, storage services, and management services.

Transient Failures Result in Busy Signals

There are several reasons for a cloud service request to fail: the requesting account is being too aggressive, an overall activity spike across all tenants, or it could be due to a hardware failure in the cloud service. In any case, the service is proactively managing access to its resources, trying to balance the experience across all tenants, and even reconfiguring itself on the fly in reaction to spikes, workload shifts, and internal hardware failures.

Cloud services have limits; check with your cloud vendor for documentation. Examples of limits are the maximum number of service operations that can be performed per second, how much data can be transferred per second, and how much data can be transferred in a single operation.

In the first two examples, operations per second and data transferred per second, even with no individual service operation at fault it is possible that multiple operations will cumulatively exceed the limits. In contrast, the third example, amount of data transfer-

red in a single operation, is different. If this limit is exceeded, it will not be due to a cumulative effect, but rather it is an invalid operation that should always be refused. Because an invalid operation should always fail, it is different from a transient failure and will not be considered further with this pattern.

Handling Busy Signals Does Not Replace Addressing Scalability Challenges

For cloud services, limits are not usually a problem except for very busy applications. For example, a Windows Azure Storage Queue is able to handle up to 500 operations per second for any individual queue. If your application needs to sustain more than 500 queue operations per second on an individual queue, this is no longer a transient failure, but rather a scalability challenge. Techniques for overcoming such a scalability challenge are covered under *Horizontally Scaling Compute Pattern* (Chapter 2) and *Auto-Scaling Pattern* (Chapter 4).

Limits in cloud services can be exceeded by an individual client or by multiple clients collectively. Whenever your use of a service exceeds the maximum allowed throughput, this will be detected by the service and your access would be subject to throttling. *Throttling* is a self-defense response by services to limit or slow down usage, sometimes delaying responses, other times rejecting all or some of an application's requests. It is up to the application to retry any requests rejected by the service.

Multiple clients that do not exceed the maximum allowed throughput *individually* can still exceed throttling limits *collectively*. Even though no individual client is at fault, aggregate demand cannot be satisfied. In this case the service will also throttle one or more of the connected clients. This second situation is known as the *noisy neighbor problem* where you just happen to be using the same service instance (or virtual machine) that some other tenant is using, and that other tenant just got real busy. You might get throttled even if, technically, you do nothing wrong. The service is so busy it needs to throttle *someone*, and sometimes that someone is you.

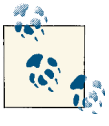
Cloud services are dynamic; a usage spike caused by a bunch of noisy neighbors might be resolved milliseconds later. Sustained congestion caused by multiple active clients who, as individuals, are compliant with rate limits, should be handled by the sophisticated resource monitoring and management capabilities in the cloud platform. Resource monitoring should detect the issue and resolve it, perhaps by spreading some of the load to other servers.

Cloud services also experience internal failures, such as with a failed disk drive. While the service automatically repairs itself by failing over to a healthy disk drive, redirecting

traffic to a healthy node, and initiating replication of the data that was on the failed disk (usually there are three copies for just this kind of situation), it may not be able to do so instantaneously. During the recovery process, the service will have diminished capacity and service calls are more likely to be rejected or time out.

Recognizing Busy Signals

For cloud services accessed over HTTP, transient failures are indicated by the service rejecting the request and usually responded to with an appropriate HTTP status code such as: *503 Service Unavailable*. For a relational database service accessed over TCP, the database connection might be closed. Other short-lived service outages may result in different error codes, but the handling will be similar. Refer to your cloud service documentation for guidance, but it should be clear when you have encountered a transient failure and documentation may also prescribe how best to respond. Handle (and log) unexpected status codes.



It is important that you clearly distinguish between busy signals and errors. For example, if code is attempting to access a resource and the response indicates it has failed because the resource does not exist or the caller does not have sufficient permissions, then retries will not help and should not be attempted.

Responding to Busy Signals

Once you have detected a busy signal, the basic reaction is to simply retry. For an HTTP service, this just means reissuing the request. For a database accessed over TCP, this may require reestablishing a database connection and then reissuing the query.

How should your application respond if the service fails again? This depends on circumstances. Some responses to consider include:

- Retry immediately (no delay).
- Retry after delay (fixed or random delay).
- Retry with increasing delays (linear or exponential backoff) with a maximum delay.
- Throw an exception in your application.

Access to a cloud service involves traversing a network that already introduces a short delay (longer when accessing over the public Internet, shorter when accessing within a data center). A *retry immediately* approach is appropriate if failures are rare and the documentation for the service you are accessing does not recommend a different approach.

When a service throttles requests, multiple client requests may be rejected in a short time. If all those clients retry quickly at the same time, the service may need to reject many of them again. A *retry after delay* approach can give the service a little time to clear its queue or rebalance. If the duration of the delay is random (e.g., 50 to 250ms), retries to the busy service across clients will be more distributed, improving the likelihood of success for all.

The least aggressive retry approach is *retry with increasing delays*. If the service is experiencing a temporary problem, don't make it worse by hammering the service with retry requests, but instead get less aggressive over time. A retry happens after some delay; if further retries are needed, the delay is increased before each successive retry. The delay time can increase by a fixed amount (*linear backoff*), or the delay time can, for example, double each time (*exponential backoff*).

Cloud platform vendors routinely provide client code libraries to make it as easy as possible to use your favorite programming language to access their platform services. Avoid duplication of effort: some client libraries may already have retry logic built in.

Regardless of the particular retry approach, it should limit the number of retry attempts and should cap the backoff. An aggressive retry may degrade performance and overly tax a system that may already be near its capacity limits. Logging retries is useful for analysis to identify areas where excessive retrying is happening.

After some reasonable number of delays, backoffs, and retries, if the service still does not respond, it is time to give up. This is both so the service can recover and so the application isn't locked up. The usual way for application code to indicate that it cannot do its job (such as store some data) is to throw an exception. Other code in the application will handle that exception in an application-appropriate manner. This type of handling needs to be programmed into every cloud-native application.

User Experience Impact

Handling transient failures sometimes impacts the user experience. The details of handling this well are specific to every application, but there are a couple of general guidelines.

The choice of a retry approach and the maximum number of retry attempts should be influenced by whether there is an interactive user waiting for some result or if this is a batch operation. For a batch operation, exponential backoff with a high retry limit may make sense, giving the service time to recover from a spike in activity, while also taking advantage of the lack of interactive users.

With an interactive user waiting, consider several retries within a small interval before informing the user that "the system is too busy right now – please try again later". The social networking service Twitter is well-known for this behavior. Consider the *Queue-Centric Workflow Pattern* (Chapter 3) for ways to decouple time-consuming work from the user interface.

When a service does not succeed within a reasonable time or number of retries, your application should take action. Though unsatisfying, sometimes passing the information back to the user is a reasonable approach, such as "the server is busy and your update will be retried in ten seconds". (This is similar to how Google Mail and Quora handle temporary network connectivity issues in their web user interfaces.)

Be careful with server-side code that ties up resources while retrying some operation, even when that code is retrying in an attempt to improve the user experience. If a busy web application has lots of user requests, each holding resources during retries, this could bump up against other resource constraints, reducing scalability.

Logging and Reducing Busy Signals

Logging busy signals can be helpful in understanding failure patterns. Robustly tracking and handling transient failures is extremely important in the cloud due to the innate challenges in debugging and managing distributed cloud applications.

Analysis of busy signal logs can lead to changes that will reduce future busy signals. For example, analysis may reveal that busy signals trend higher when accessing a cloud database service. Remember, the cloud provides the illusion of infinite resources, but this does not mean that each resource has infinite capacity. To access more capacity, you must provision more instances. When one database instance is not enough, it may be time to apply *Database Sharding Pattern* (Chapter 7).

Testing

It is common to test cloud applications in non-cloud environments. Code that runs in a development or test environment, especially at lower-than-production volumes or with dedicated hardware, may not experience the transient failures seen in the cloud. Be sure to test and load test in an environment as close to production as possible.

It is becoming more common for companies to test against the production environment because it is the most realistic. For example, load testing against production, though perhaps at non-peak times. Netflix goes even further by continually stressing their production (cloud) environment with errors using a home-grown tool they call Chaos Monkey. To ensure they can handle any kind of disruption, Chaos Monkey continually and randomly turns off services and reboots servers in the production environment.

Example: Building PoP on Windows Azure

The Page of Photos (PoP) application (which was described in the Preface) uses a number of Windows Azure cloud services. Two of these are Windows Azure SQL Databases for storing account profile information and Windows Azure Storage blobs for storing photos. Because these are cloud services, PoP code is written to handle transient failures.

The Windows Azure Platform includes software libraries for use with a variety of programming environments such as C#, Java, Node.js, Python, and mobile devices. These libraries all simplify accessing blobs by automatically detecting transient failures and retrying up to three times. There are a number of other predefined retry behaviors available, and it is also possible to create a custom retry policy. Most applications won't need anything beyond the default behavior.

Busy Signals in the Real World

How frequently do retries happen in practice? Here are some real numbers gathered in running an upload utility that saturated a network connection for an extended period of time (measured in days), pushing data over the public Internet into Windows Azure Blob storage. A custom retry policy was used to implement exponential backoff while also logging each retry attempt. During the uploading of nearly a million files averaging four megabytes in size, around 1.8% of the *files* required at least one retry. Note that because the files were so large, each file upload involved many storage operations, so retries on storage operations were needed much less frequently than 0.1% of the time. However, because logging was at the granularity of a file upload level, exact statistics are not available. Further, many factors can impact retry behavior, such as competition for the network resources locally and network connectivity across the public Internet. Your results will vary.

Applications accessing Windows Azure SQL Databases use the same TCP protocol used for accessing SQL Server. However, a more robust approach to transient failure detection and retry logic is needed. This comes in the form of a library known as the Transient Fault Handling Application Block, also known as Topaz. Like the retry support in the

libraries mentioned previously for blob access, Topaz has some predefined retry behaviors, but also a rich model for customization. The easiest way to take advantage of Topaz features when writing database access code is to replace standard database objects with the transient-failure-aware equivalents that ship with Topaz. For example, use a `ReliableSqlConnection` object in place of the standard .NET `SqlConnection` object.

Some SQL Database transient failures are due to throttling and inform you of this by returning a well-known error code. Sometimes SQL Database will drop your database connection, requiring that you reconnect.

Expect Your Database Connection to Drop

Why would SQL Database drop your connection? When you connect, behind the scenes it is not a single instance but rather a cluster of three collaborating servers. This provides multiple benefits. One benefit is resilience to disk drive failure as each byte written to SQL Database is written to all three servers within the cluster. Another benefit is that, as a multitenant service, SQL Database can distribute (and redistribute) load across servers to keep it balanced. These benefits come at a cost. If SQL Database chooses your connection to move to another of the three servers in the cluster, it needs to first disconnect you; when you reconnect, you will connect to your newly assigned cluster node, though you won't be able to tell the difference.

While the focus here has been on using Topaz with SQL Database, it also supports Windows Azure Storage, Windows Azure Caching, and Windows Azure Service Bus, including advanced retry possibilities if you need them.

Related Chapters

- *Queue-Centric Workflow Pattern* (Chapter 3)
- *Multitenancy and Commodity Hardware Primer* (Chapter 8)
- *Node Failure Pattern* (Chapter 10)

Summary

Handling transient failures is essential for building reliable cloud-native applications. Using the Busy Signal Pattern, your application can detect and handle transient failures appropriately. Further, your approach can be tuned for batch or interactive user scenarios.

It may be difficult to test your application's response to transient failure conditions if running on non-cloud hardware or with an unrealistically light load.

CHAPTER 10

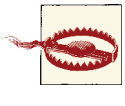
Node Failure Pattern

This pattern focuses on how an application should respond when the compute node on which it is running shuts down or fails.

This pattern reflects the perspective of application code running on a node (virtual machine) that is shut down or suddenly fails due to a software or hardware issue. The application has three responsibilities: prepare the application to minimize issues when nodes fail, handle node shutdowns gracefully, and recover once a node has failed.

Some common reasons for shutdown are unresponsive application due to application failure, routine maintenance activities managed by the cloud vendor, and auto-scaling activities initiated by the application. Failures might be caused by hardware failure or an unhandled exception in your application code.

While there are many reasons for a node shutdown or failure, we can still treat them uniformly. Handling the various forms of failure is sufficient; all shutdown scenarios will also be handled. The pattern name derives from the more encompassing node failures.



Applications that do not handle node shutdowns and failures will be unreliable.

Context

The Node Interruption Pattern is effective in dealing with the following challenges:

- Your application is using the Queue-Centric Workflow Pattern and requires *at-least-once processing* for messages sent across tiers

- Your application is using the Auto-Scaling Pattern and requires graceful shutdown of compute nodes that are being released
- Your application requires graceful handling of sudden hardware failures
- Your application requires graceful handling of unexpected application software failures
- Your application requires graceful handling of reboots initiated by the cloud platform
- Your application requires sufficient resiliency to handle the unplanned loss of a compute node without downtime

All of these challenges result in interruptions that need to be addressed and many of them have overlapping solutions.

Cloud Significance

Cloud applications will experience failures that result in node shutdowns. Cloud-native applications expect these and respond appropriately when notified of shutdowns by the cloud platform.

Impact

Availability, User Experience

Mechanics

The goal of this pattern is to allow individual nodes to fail while the application as a whole remains available. There are several categories of failure scenarios to consider, but all of them share characteristics for preparing for failure, handling node shutdown (when possible), and then recovering.

Failure Scenarios

When your application code runs in a virtual machine (especially on commodity hardware and on a public cloud platform), there are a number of scenarios in which a shutdown or failure could occur. There are other potential scenarios, but from the point of view of the application, the scenarios always look like one of those listed in Table 10-1.

Table 10-1. Node Failure Scenarios

Scenario	Initiated By	Advanced Warning	Impact
Sudden failure + restart	Sudden hardware failure	No	Local data is lost

Scenario	Initiated By	Advanced Warning	Impact
Node shutdown + restart	Cloud platform (vacate failing hardware, software update)	Yes	Local data may be available
Node shutdown + restart	Application (application update, application bug, managed reboot)	Yes	Local data is available
Node shutdown + termination	Application (node released such as via auto-scale)	Yes	Local data is lost

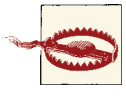


Table 10-1 is correct for the Windows Azure and Amazon Web Services platforms. It is not correct for every platform, though the pattern is still generally applicable.

The scenarios have a number of triggers listed; some provide advanced warning, some don't. Advanced warning can come in multiple forms, but amounts to a proactive signal, sent by the cloud platform that allows the node to gracefully shut down. In some cases, local data written to the local virtual machine disk is available after the interruption, sometimes it is lost. How do we deal with this?

Scenarios can be initiated by hardware or software failures, by the cloud platform, or by your application.

Treat All Interruptions as Node Failures

None of the scenarios are predictable from the point of view of the individual node. Further, the application code does not know which scenario it is in, but it can handle all of the scenarios gracefully if it treats them all as failures.

All application compute nodes should be ready to fail at any time.

Given that failure can happen at any time, your application should not use the local disk drive of the compute node as the system of record for any business data. As mentioned in Table 10-1, it can be lost. Remember that an application using stateless nodes is not the same as a stateless application. Application state is persistent in reliable storage, not on the local disk of an individual node.

By treating all shutdown and failure scenarios the same, there is a clear path for handling them: maintain capacity, handle node shutdowns, shield users when possible, and resume work-in-progress after the fact.

Maintain Sufficient Capacity for Failure with N+1 Rule

An application prepares first by assuming node failures will happen, then taking proactive measures to ensure failures don't result in application downtime. The primary proactive measure is to ensure sufficient capacity.

How many node instances are needed for high availability? Apply the *N+1 rule*: If N nodes are needed to support user demand, deploy N+1 nodes. One node can fail or be interrupted without impact to the application. It is especially important to avoid any single points of failure for nodes directly supporting users. If a single node is required, deploy two.

The *N+1 rule* should be considered and applied independently for each type of node. For example, some types of node can experience downtime without anyone noticing or caring. These are good candidates for not applying the *N+1 rule*.

A buffer more extensive than a single node may be needed, though rarely, due to an unusual failure. A severe weather-related incident could render an entire data center unavailable, for example. A top-of-rack switch failure is discussed in the *Example* and cross-data center failover is touched on in *Multisite Deployment Pattern* (Chapter 15).

There is a time lag between the failure occurrence and the recognition of that failure by the cloud platform monitoring system. Your service will run at diminished capacity until a new node has been fully deployed. For nodes configured to accept traffic through the cloud platform load balancer, traffic will continue to be routed there; once the failed node is recognized it will be promptly removed from the load balancer, but there will still be more time until the replacement node is booted up and added to the load balancer. See *Auto-Scaling Pattern* (Chapter 4) for a discussion of borrowing already-deployed resources from less critical functions to service more important ones during periods of diminished capacity.

Handling Node Shutdown

We can responsibly handle node shutdown, but there is no equivalent handling for sudden node failure because the node just stops in an instant. Not all hardware failures result in node failure: the cloud platforms are adept at detecting signals that hardware failure is imminent and can preemptively initiate a controlled shutdown to move tenants to different hardware.

Some node failures are also preventable: is your application code handling exceptions? At a minimum, do this for logging purposes.

Regardless of the reason the node is shutting down, the goals are to allow in-progress work to complete, gather operational data from that node before shutdown completes, and do so without negatively impacting the user experience.

Cloud platforms provide signals to let running applications know that a node is either about to be, or is in the process of being, shut down. For example, one of the signals provided by Amazon Web Services is an advanced alert, and one of the signals provided by Windows Azure is to raise an event within the node indicating that shutdown has begun.

Node shutdown with minimal impact to user experience

Cloud platforms managing both load balancing and service shutdown stop sending web traffic to nodes they are in the process of shutting down. This will prevent many types of user experience issues, as new web requests will be routed to other instances. Only those pending web requests that will not be complete by the time the service shuts down are problematic. Some applications will not have any problem as all requests are responded to nearly instantly.

If your application is using sticky sessions, new requests will not always be routed to other instances. This topic was covered in *Horizontally Scaling Compute Pattern* (Chapter 2). However, cloud-native applications avoid sticky sessions.

It is possible that a node will begin shutting down while some work visible to a user is in progress. The longer it takes to satisfy a web request, the more of a problem this can be. The goal is to avoid having users see server errors just because they were accessing a web page when a web node shuts down while processing their request. The application should wait for these requests to be satisfied before shutting down. How to best handle this is application-, operating system-, and cloud platform-specific; an example for a Windows Cloud Service running on Windows Azure is provided in the *Example*.

Failures that result in sudden termination can cause user experience problems if they happen in nodes serving user requests. One tactic for minimizing this is to design the user interface code to retry when there are failures, as described in *Busy Signal Pattern* (Chapter 9). Note that the busy signal here is not coming from a cloud platform service, but the application's own internal service.

Node shutdown without losing partially completed work

The node should immediately stop pulling new items from the queue once shutdown is underway and let current work finish if possible.

Queue-Centric Workflow Pattern (Chapter 3) describes processing services that can sometimes be lengthy. If processing is too lengthy to be completed before the node is terminated, that node has the option of saving any in-progress work outside of the local node (such as to cloud storage) in a form that can be resumed.

Node shutdown without losing operational data

Web server logs and custom application logs are typically written to local disk on individual nodes. This is a reasonable approach, as it makes efficient use of available local resources to capture this log data.

As described in *Horizontally Scaling Compute Pattern* (Chapter 2), part of managing many nodes is the challenge of consolidating operational data. Usually this data is collected periodically. If the collection process is not aware that a node is in the process of shutdown, that data may not be collected in time, and may be lost. This is your opportunity to trigger collection of that operational data.

Recovering From Node Failure

There are aspects to recovering from node failure: maintaining a positive user experience during the failure process, and resuming any work-in-progress that was interrupted.

Shielding interactive users from failures

Some node instances support interactive users directly. These nodes might be serving up web pages directly or providing web services that power mobile apps or web applications. A positive user experience would shield users from the occasional failure. This happens in a few ways.

If a node fails on the server, the cloud platform will detect this and stop sending traffic to that node. Once detected, service calls and web page requests will be routed to healthy node instances. You don't need to do anything to make this happen (other than follow the *N+1 rule*), and this is sometimes good enough.

However, to be completely robust, you should consider the small window of exposure to failure that can occur while in the middle of processing a request for a web page or while executing a service call. For example, the disk drive on the node may suddenly fail. The best response to this edge case is to place retry logic on the client, basically applying *Busy Signal Pattern* (Chapter 9). This is straightforward for a native mobile app or a single-page web application managing updates through web service calls (Google Mail is an example of a single-page web application that handles this scenario nicely). For a traditional web application that redisplays the entire page each time, it is possible that users could see an error surface through their web browser. This is out of your control. It will be up to the users to retry the operation.

Don't Be Overprotective

The strategy of shielding users from errors only makes sense up to a point. Eventually you will need to communicate errors to the user, though "service is not available, please retry

in a couple of minutes" is sometimes the best we can do. Further, make sure any retry policies are appropriate for interactive users; in other words, a policy that retries 10 times with a 30-second sleep in between may be fine for a nightly batch process, but totally inappropriate when supporting an interactive user.

Resuming work-in-progress on backend systems

In addition to user experience impact, sudden node failure can interrupt processing in the service tier. This can be robustly handled by building processes to be idempotent so they can safely execute multiple times on the same inputs. Successful recovery depends on nodes being stateless and important data being stored in reliable storage rather than on a local disk drive of the node (assuming that disk is not backed by reliable storage). A common technique for cloud-native applications is detailed in *Queue-Centric Workflow Pattern* (Chapter 3), which covers restarting interrupted processes as well as saving in-progress work to make recovery faster.

Example: Building PoP on Windows Azure

Page of Photos (PoP) application (which was described in the Preface) is architected to provide a consistently reliable user experience and to not lose data. In order to maintain this through occasional failures and interruptions, PoP prepares for failure, handles role instance (node) shutdowns gracefully, and recovers from failures as appropriate.

Preparing PoP for Failure

PoP is constantly increasing and decreasing capacity to take advantage of the cost savings; we only want to pay for the capacity needed to run well, and no more. PoP also does not wish to sustain downtime or sacrifice the user experience due to diminished capacity in the case of an occasional failure or interruption.

N+1 rule

The *N+1 rule* is honored for roles that directly serve users, because user experience is very important. However, because no users are directly impacted by occasional interruptions, PoP management made the business decision to not apply the *N+1 rule* for worker roles which make up the service tier.

These decisions are accounted for in PoP auto-scaling rules.

Windows Azure fault domains

Windows Azure (through a service known as the Windows Azure Fabric Controller) determines where in the data center to deploy each role instance of your application

within specific constraints. The most important of these constraints for failure scenarios is a fault domain. A *fault domain* is a potential single point of failure (SPoF) within a data center, and any roles with a minimum of two instances are guaranteed to be distributed across at least two fault domains.

The discerning reader will realize that this means up to half of your application's web and worker role instances can go down at once (assuming two fault domains). While it is possible, it is unlikely. The good news is that if it does happen, the Fabric Controller begins immediate remediation, although your application will run with diminished capacity during recovery.

Auto-Scaling Pattern (Chapter 4) introduced the idea of internally throttling some application features during periods of diminished or insufficient resources. This is another scenario where that tactic may be useful. Note that you may not want to auto-scale in response to such a failure, but rather allow the Fabric Controller to recover for you.

For the most part, a hardware failure results in the outage of a single role instance. The *N+1* rule accounts for this possibility.

PoP makes the business decision that N+1 is sufficient, as described previously.

When N + 1 Is Not Enough

If a failure impacts more than a single hardware node (for example, a failure at the rack level), N+1 may not be enough, depending on your business requirements. If this level of potential (albeit rare) downtime is too much, consider even more capacity, depending on how many fault domains are in effect. Currently, all Windows Azure cloud services run with two fault domains and (as of this writing) the number cannot be changed. Extra capacity for critical times can be scheduled as a proactive auto-scaling task. Also consider *Multisite Deployment Pattern* (Chapter 15).

Note that fault domains only apply with unexpected hardware failures. For operating system updates, hypervisor updates, or application-initiated upgrades, downtime is distributed across upgrade domains.

Upgrade domains

Conceptually similar to fault domains, Windows Azure also has *upgrade domains* that provide logical partitioning of role instances into groups (by default, five) which are considered to be independently updatable. This goes hand-in-hand with the *in-place upgrade* feature, which upgrades your application in increments, one upgrade domain at a time.

If your application has N upgrade domains, the Fabric Controller will manage the updates such that 1/N role instances will be impacted at a time. After these updates complete, updating the next 1/N role instances can begin. This continues until the application is completely updated. Progressing from one upgrade domain to the next can be automated or manual.

Upgrade domains are useful to the Fabric Controller since it uses them during operating system upgrades (if the customer has opted for automatic updates), hypervisor updates, emergency moves of role instances from one machine or rack to another, and so forth. Your application uses upgrade domains if it updates itself using the *in-place upgrade* process where the application is upgraded one upgrade domain at a time. You may choose to manually approve the progression from one upgrade domain to the next, or have the Fabric Controller proceed automatically.

Multiple Concurrent Versions

Cloud-native applications commonly have a single deployment with mixed releases (and features) across the role instances. This staged upgrade can be managed using upgrade domains with an in-place upgrade. Another way to look at this: an application upgrade does not happen at a particular time, rather it spans time. During that time span the application is running two versions. Two customers visiting the application at the same time may be routed to different versions. If you wish to avoid deploying multiple concurrent versions, look into Windows Azure's VIP Swap feature.

What is the right number of upgrade domains for my application? There is a tradeoff: more upgrade domains take longer to roll out, while fewer upgrade domains increase the level of diminished capacity. For PoP, the default of five upgrade domains works well.

Handling PoP Role Instance Shutdown

Graceful role shutdown avoids loss of operational data and degradation of the user experience.

When we instruct Windows Azure to release a role instance, there is a defined process for shutdown; it does not shut off immediately. In the first step, Windows Azure chooses the role instance that will be terminated

You don't get to decide which role instances will be terminated.

First, let's consider terminating a web role instance.

Web role instance shutdown

Once a specific web role instance is selected for termination, Windows Azure (specifically the Fabric Controller) removes that instance from the load balancer so that no new requests will be routed to it. The Fabric Controller then provides a series of signals to the running role that a shutdown is coming. The last of these signals is to invoke the role's `OnStop` method, which is allowed five minutes for a graceful shutdown. If the instance finishes the cleanup within five minutes, it can return from `OnStop` at that point; if your instance does not return from `OnStop` within five minutes, it will be forcibly terminated. Only after `OnStop` has completed does Azure consider your instances to be released for billing purposes, providing an incentive for you to make `OnStop` exit as efficiently as possible. However, there are a couple of steps you could take to make the shutdown more graceful.

Although the first step in the shutdown process is to remove the web role from the load balancer, it may have existing web requests being handled. Depending on how long it takes to satisfy individual web requests, you may wish to take proactive steps to allow these existing web requests to be satisfied before allowing the shutdown to complete. One example of a slower-than-usual operation is a user uploading a file, though even routine operations may need to be considered if they don't complete quickly enough. A simple technique to ensure your Web Role is gracefully shutting down is to monitor an IIS performance counter such as *Web Service\Current Connections* in a loop inside your `OnStop` method, continuing to termination only when it has drained to zero.

Application diagnostic information from role instances is logged to the current virtual machine and collected by the Diagnostics Manager on a schedule of your choosing, such as every ten minutes. Regardless of your schedule, triggering an unscheduled collection within your `OnStop` method will allow you to wrap up safely and exit as soon as you are ready. An on-demand collection can be initiated using Service Management features.

Worker role instance shutdown

Gracefully terminating a worker role instance is similar to terminating a web role instance, although there are some differences. Because Azure does not load balance PoP worker role instances, there are no active web requests to drain. However, triggering an on-demand collection of application diagnostic information does make sense, and you would usually want to do this.

Although PoP worker role instances do not have public-facing endpoints, Windows Azure does support it, so applications may have active connections when instance shutdown begins. A worker role can define public-facing endpoints that will be load balanced. This feature is useful in running your own web server such as Apache, Mongoose, or Nginx. It may also be useful for exposing self-hosted WCF endpoints. Further, public endpoints are not limited to HTTP, as TCP and UDP are also supported.

A worker role instance will want to stop pulling new items from the queue for processing as soon as possible once shutdown is underway. As with the web role, instance shutdown signals such as `OnStop` can be useful; a worker role could set a global flag to indicate that the role instance is shutting down. The code that pulls new items from the queue could check that flag and not pull in new items if the instance is shutting down. Worker roles can use `OnStop` as a signal, or hook an earlier signal such as the `OnStopping` event.

If the worker role instance is processing a message pulled from a Windows Azure Storage queue and it does not complete before the instance is terminated, that message will time out and be returned to the queue so it can be pulled again. (Refer to *Queue-Centric Workflow Pattern* (Chapter 3) for details.) If there are distinct steps in processing, the underlying message can be updated on the queue to indicate progress. For example, PoP processes newly uploaded photos in three steps: extract geolocation information from the photo, create two sizes of thumbnail, and update all SQL Database references. If two of these steps were completed, but not the third, it is indicated by a `LastCompletedStep` field in the message object that was pulled from the queue, and that message object can be used to update the copy on the queue. When this message is pulled again from the queue (by a subsequent role instance), processing logic will know to skip the first two steps because `LastCompletedStep` field is set to two. The first time a message is pulled, the value is set to zero.

Use controlled reboots

Sometimes you may wish to reboot a role instance. The controlled shutdown that allows draining active work is supported if reboots are triggered using the Reboot Role Instance operation in the Windows Azure Service Management service.

If other means of initiating a reboot are used, such as directly on the node using Windows commands, Windows Azure is not able to manage the process to allow for a graceful shutdown.

Recovering PoP From Failure

Queue-Centric Workflow Pattern (Chapter 3) details recovering from a long-running process.

Otherwise, there is usually little to do in recovering a stateless role instance, whether a web role or worker role.

Some user experience glitches are addressed in PoP because client-side JavaScript code that fetches data (though AJAX calls to REST services) implements *Busy Signal Pattern* (Chapter 9). This code will retry if a service does not respond, and the retry will eventually be load balanced to a different role instance that is able to satisfy the request.

Related Chapters

- *Queue-Centric Workflow Pattern* (Chapter 3)
- *MapReduce Pattern* (Chapter 6)
- *Multitenancy and Commodity Hardware Primer* (Chapter 8)
- *Busy Signal Pattern* (Chapter 9)
- *Multisite Deployment Pattern* (Chapter 15)

Summary

Failure in the cloud is commonplace, though downtime is rare for cloud-native applications. Using the Node Failure Pattern helps your application prepare for, gracefully handle, and recover from occasional interruptions and failures of compute nodes on which it is running. Many scenarios are handled with the same implementation.

Cloud applications that do not account for node failure scenarios will be unreliable: user experience will suffer and data can be lost.

CHAPTER 11

Network Latency Primer

This basic primer explains network latency and why delays due to network latency matter.

The time it takes to transmit data across a network is known as *network latency*. Network latency slows down our application.

While individual networking devices like routers, switches, wireless access points, and network cards all introduce latencies of their own, this primer blends them all together into a bigger picture view, the total delay experienced by data having to travel over the network.

As cloud application developers, we can decrease the impact of network latency through caching, compression, moving nodes closer together, and shortening the distance between users and our application.

Network Latency Challenges

Highly scalable and high performing (even infinitely fast!) servers do not guarantee that our application will perform well. This is due to the main performance challenge that lies outside of raw computational power: movement of data. Transmitting data across a network does not happen instantly and the resultant delay is known as *network latency*.

Network latency is a function of distance and bandwidth: how far the data needs to travel and how fast it moves. The challenge is that any time compute nodes, data sources, and end users are not all using a single computer, network latency comes into play; the more distribution, the greater the impact. Network quality plays an important role, although it is one you might not be able to control; quality may vary as users connect through networks from disparate locations.

Application performance and scalability will suffer if it takes too long for data to get to the client computer. The effects of network latency can also vary based on the user's geographical location: to some users of the system, it may seem blazingly fast, while to other users it may seem as slow as cold molasses (which means *really slow*).

Admittedly, understanding the actual distance traveled by the data and the effective bandwidth can be challenging. The actual path traveled is not the same as the ideal great circle path you might imagine from a map. The path from router to router is jagged, and indeed may even be impacted by the capability of individual routers, and individual legs of the route may have different bandwidths. And it can vary over time.

A simple way to estimate effective bandwidth (at some point in time) is to use ping. *Ping* is a simple but useful program that calculates the time it takes to travel from one point on the Internet to another point, by actually sending network packets, counting the nodes along the way, and showing how long it takes the packets to get to the destination and back. Some measured ping times are shown in Table 11-1. These pings originated from Boston which is in the eastern USA. The fastest networks today use fiber optic cable, which supports data transmission at around 66% of the speed of light (186,000 miles/sec × 66% = 122,760 miles/sec). As you can see from Table 11-1, shorter distances are dominated by factors other than pure distance traveled, and none are close to the theoretical maximum speed. HTTP and TCP traffic will be slower than pings. Network latency impact can add up quickly, especially as web pages get heavier with more objects to download. In *An Analysis of Application Performance Data and Trends* (Jan 2012), Compuware reports that the average (yes, *average*) page download size for a non-mobile news site is more than a megabyte and 790 KB for a non-mobile travel site.

Table 11-1. Ping Times from Boston – distance matters

Domain	Location	Elapsed Time each way (ms)	Approximate Distance (miles)	Speed (miles/sec)
www.bu.edu	Boston, USA	17	10	1,000
www.gsu.edu	Atlanta, USA	75	900	12,000
www.usc.edu	Los Angeles, USA	51	2600	52,000
www.cam.ac.uk	Cambridge, UK	50	3300	67,000
www.msu.ru	Moscow, RU	81	4500	56,000
www.u-tokyo.ac.jp	Tokyo, JP	107	6700	63,000
www.auckland.ac.nz	Auckland, NZ	115	9000	79,000

Doing the Math

How much data transfer can I get for my bandwidth? Let's do some math. Consider a T1 connection that provides 1.544 Mb/sec of bandwidth. Translating from kilo**bits** (Kb) to kilo**bytes** (KB) we end up with about 193 KB/sec (1544/8=193 kilo**bytes** per second, ig-

noring errors, retries, speed degradation, and assuming no other uses of the connection). Uploading a full CD-ROM (~660 MB) would take nearly an hour. Uploading a full DVD (~4.7 GB) would take around 7 hours. These calculations assume ideal network conditions, which of course you will never have.

Reducing Perceived Network Latency

The *perceived network latency*, the network latency as experienced by a user, can be reduced through techniques such as:

- Data compression
- Background processing, where screen updates don't happen until the data arrives (though not actually faster, it may improve a user's subjective experience, as with single page web applications)
- Predictive fetching, where data is loaded in anticipation of need (as with map tiles in a mapping app, although it is possible that not all the map tiles will be referenced)

Eventual consistency is another tool we can use, if we can serve users with slightly stale data. These are reasonable approaches for reducing the *impact* of network latency, but do not *change* the network latency. They are essentially the same for cloud-native applications as they are for non-cloud applications.

Reducing Network Latency

We can reduce network latency by:

- Moving application closer to users
- Moving application data closer to users
- Ensuring nodes within our application are close together

These reduction techniques are the topics of the Colocate, Valet Key, CDN, and Multisite Deployment patterns.

Summary

A comprehensive strategy for dealing with network latency will use multiple strategies. One set of strategies focuses on reducing the perceived network latency. Another set of strategies focuses on actually reducing network latency by shortening the distance between users and the instances of our application.

CHAPTER 12
Colocate Pattern

This basic pattern focuses on avoiding unnecessary network latency.

Communication between nodes is faster when the nodes are close together. Distance adds network latency. In the cloud, "close together" means in the same data center (sometimes even closer, such as on the same rack).

There are good reasons for nodes to be in different data centers, but this chapter focuses on ensuring that nodes that should be in the same data center actually are. Accidentally deploying across multiple data centers can result in terrible application performance and unnecessarily inflated costs due to data transfer charges.

This applies to nodes running application code, such as compute nodes, and nodes implementing cloud storage and database services. It also encompasses related decisions, such as where log files should be stored.

Context

The Colocation Pattern effectively deals with the following challenges:

- One node makes frequent use of another node, such as a compute node accessing a database
- Application deployment is basic, with no need for more than a single data center
- Application deployment is complex, involving multiple data centers, but nodes within each data center make frequent use of other nodes, which can be colocated in the same data center

In general, resources that are heavily reliant on each other should be colocated.

A multitier application generally has a web or application server tier that accesses a database tier. It is often desirable to minimize network latency across these tiers by colocating them in the same data center. This helps maximize performance between these tiers and can avoid the costs of cloud provider data transmission.

This pattern is typically used in combination with the Valet Key and CDN Patterns. Reasons to deviate from this pattern, such as proximity to consumers and overall reliability, are discussed in *Multisite Deployment Pattern* (Chapter 15).

Cloud Significance

Public cloud platforms typically offer many worldwide data center locations to which applications can be easily deployed. These data centers span continents, and sometimes multiple data centers are available within the same continent. An application can be easily deployed to and use the cloud platform services in any of these data centers. This multi-data center flexibility brings great advantages, but also introduces the risk that application code and services will be unnecessarily distributed across multiple data centers, resulting in extra costs and a degraded user experience.

Impact

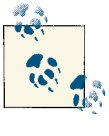
Cost Optimization, Scalability, User Experience

Mechanics

When you think about it, this may seem an obvious pattern, and in many respects it is. Depending on the structure of your company's hardware infrastructure (whether a private data center or rented space), it may have been very difficult to do anything *other than* colocate databases and the servers that accessed them.

With public cloud providers, multiple data centers are typically offered across multiple continents, sometimes with more than one data center per continent or region. If you plan to deploy to a single data center, there may be more than one reasonable choice. This is good news and bad news, since it is possible (and easy) to choose any data center as a deployment target. This makes it possible to deploy a database to one data center, while the servers that access the database are deployed to a different data center.

The performance penalty of a split deployment can be severe for a data-intensive application.



Hadoop-based "big data" applications tend to be data-intensive in the extreme and require that data and compute be colocated. See *MapReduce Pattern* (Chapter 6).

Automation Helps

Enforcing colocation is really not a technological problem, but rather a process issue. However, it can be mitigated with automation.

You will want to take proactive steps to avoid accidentally splitting a deployment across data centers. Automating deployments into the cloud is a good practice, as it limits human error from repetitive tasks. If your application spans multiple data centers but each site operates essentially independently, add checks to ensure that data access is not accidentally spanning data.

Outside of automation, your cloud platform may have specific features that make co-location mistakes less likely.

Cost Considerations

Operations will generally be less expensive if your databases and the compute resources that access them are in the same data center. There are cost implications when splitting them.

As of this writing, the Amazon Web Services and Windows Azure platforms do not charge for network traffic to enter a data center, but do charge for network traffic leaving a data center (even if it is to another of their data centers). There are no traffic charges when data stays within a single data center.

Non-Technical Considerations

There may be non-technical influences on application architecture that result in databases being stored at a different location than the compute resources that access them. This topic is considered further in *Multisite Deployment Pattern* (Chapter 15).

Example: Building PoP on Windows Azure

When the Page of Photos (PoP) application (which was described in the Preface) was first developed, it made sense to deploy it and use cloud services inside of a single data center.

Windows Azure allows you to specify the target data center for any resource that is deployable to a specific data center. As examples, the target data center can be specified for code deployment (such as Web Roles, Worker Roles, or Virtual Machines) and cloud services (such as Windows Azure Storage, SQL Database, and more). When such resources will be used together, they should be colocated in the same data center.

Affinity Groups

Windows Azure goes a step further for some resources, supporting affinity groups. An *affinity group* is a logical grouping of resources tied to a data center. You can provide a custom name for an affinity group, using a name that makes sense for your business and is not simply a generic data center name. This is an example of a cloud platform feature that can help avoid colocation mistakes.

In the case of PoP, if we deploy to a single North American data center, we can have an affinity group called *PoP-North America-Production* for our production data center. Upon creation, the decision is made as to which North American data center will be chosen, which removes this as a decision point for any downstream deployments that use the *PoP-North America-Production* affinity group.

As of this writing, not all Windows Azure resources support affinity groups; currently only Windows Azure Compute and Windows Azure Storage are supported. Most notably, SQL Database is not supported, although it can still be placed in the same data center as the other resources.

While affinity groups are tied to a specific data center, they also provide a hint to Windows Azure that allows for further local optimization for supported resource types. Not only are they in the same data center, but your Windows Azure Storage, the code accessing it from web, and the worker roles are even closer together, with fewer router hops and less distance for data to traverse. This further reduces network latency.

Using the same affinity group across storage accounts and cloud services will ensure that they are all colocated in the same data center.

Operational Logs and Metrics

You will also likely be gathering operational log files with Windows Azure Diagnostics (WAD) and Windows Azure Storage Analytics, available with Windows Azure Storage accounts. Apply the same affinity group to storage as you apply to compute instances.

It is a best practice to persist WAD into a special operational storage account (different from other storage accounts within your production system) both to minimize potential for contention and to make it easier to manage access to logs and metrics. Depending on the specific type of data items, individual diagnostic values are stored in either Windows Azure Blob or Windows Azure Table storage. Use the same affinity group for WAD storage to ensure it is stored in the same data center as the rest of the application.

Windows Azure Storage Analytics data are stored alongside the regular data, and are not stored in a separate storage account, so colocation is automatic.

More details about the capabilities of Windows Azure Diagnostics and Windows Azure Storage Analytics can be found in *Horizontally Scaling Compute Pattern* (Chapter 2) in the *Example* section under *Operational Logs and Metrics*. Both features support allow applications to set a basic data retention policy that Windows Azure Storage uses to automatically purge data.

When colocation is not possible due to technical or business reasons, Windows Azure has some services that can help. These services are mentioned in the *Example* section of *Multisite Deployment Pattern* (Chapter 15).

Related Chapters

- *MapReduce Pattern* (Chapter 6)
- *Network Latency Primer* (Chapter 11)
- *Valet Key Pattern* (Chapter 13)
- *CDN Pattern* (Chapter 14)
- *Multisite Deployment Pattern* (Chapter 15)

Summary

The simplest way to get started in the cloud is to colocate nodes, usually all in a single data center. This is appropriate for many applications, and should be the usual configuration. Only deviate for good reason, and avoid the mistake of accidental deployment across more than one data center, including for storage of operational data.

CHAPTER 13

Valet Key Pattern

This pattern focuses on efficiently using cloud storage services with untrusted clients.

This pattern loosely models the use of valet keys from the real world. Valet keys are useful when you are willing to trust a valet parking attendant to park your car, but don't want to also give them access to areas in the car not needed for this purpose, such as the glove compartment. This pattern enables specifying that a user of your application is allowed to access very specific areas within your cloud storage account, with specific permissions, and for a limited amount of time. You can issue as many cloud storage valet keys as you like and they can all be different.

Cloud storage services simplify securely transferring data directly between untrusted clients and to secure data storage, without the data needing to pass through a trusted intermediate layer (the web tier in this case) to implement security. Both uploads and downloads are supported.

Going directly to the final storage location eliminates the need for data to unnecessarily pass through an intermediary, so these operations will be faster with lower latency, while reducing the load on the web tier. Because this pattern avoids load on the web tier, it will result in needing less capacity, which may further result in needing fewer web server nodes (thus saving money). Realize, however, that this pattern also requires the ability to securely allow access to cloud storage, typically by issuing temporary access URLs. Depending on how this is done, it may necessitate the creation of a web service, which may require some infrastructure of its own. Generally, however, the performance, scalability, and cost savings favor the use of this pattern.

Context

The Valet Key Pattern is effective in dealing with the following challenges:

- Application data files are stored in cloud storage, which clients need to access
- Application data files are stored in cloud storage, which clients need to access on a case-by-case basis
- Application supports clients uploading data that will end up in cloud storage

This pattern is useful when reading and writing data from mobile apps, desktop apps, and during server-to-server communication. Reading data from a web application is also a very common use, though due to security limitations imposed by web browsers it is much trickier to write data to blob storage from a web application; however, see the appendix as it is possible in some cases using emerging HTML5 standards. Still, many web browsers do not support this scenario today.

Example of practical uses are many. A for-pay video or training site can allow access to a video for 24 hours. A photo sharing site can allow a user to upload a new photo or video directly into cloud storage. An enterprise can securely and efficiently share assets with employees or partners who are outside the firewall. A backup service running on a consumer desktop can efficiently interact directly with cloud storage, while only ever having access to files for that single user. A translation service can allow customers to upload text, audio, and video files; translated can be delivered back through the same secure means.

Cloud Significance

This pattern is possible because cloud services mask significant technical complexity by offering an easy-to-use service to applications. Additionally, software libraries supporting a broad range of clients further simplifies client coding.

The cloud platform data storage services are optimized for reading and writing files of all sizes at very high scale, shielding the application's web tier from needing to provide the bandwidth and computational power to handle this same data.

Some applications use the Gatekeeper Pattern, which does exactly what this pattern avoids: move all data through an intermediary in order to control access. Both patterns can be used securely, though this pattern offers efficiency and scalability benefits.

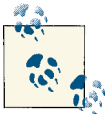
Impact

Scalability, User Experience

Mechanics

Access to cloud storage services is a privileged operation. Typically, you will allow only *trusted subsystems* within your application to have unfettered access to your cloud storage accounts, where trusted subsystems are running entirely under your control on the server.

Securing and managing access to data is always a high priority, but there is also a tension between security and efficiency; we want to remain secure, but would like to do so with the most efficient use of resources that allows us to deliver an optimized user experience. This pattern focuses on securely using blob storage while also reading and writing data with maximum efficiency. This is illustrated in Figure 13-1, which shows the flow of files directly between the client and the cloud storage service. The data flow is as efficient as possible, and in particular does not need to flow through the web tier of the application.



Using vendor-neutral terms can be challenging. A *blob* is a *binary large object* in the sense that's been used in the industry for many years and is commonly used with relational databases. Basically, a blob is a file. Windows Azure Storage happens to also call this concept a *blob*, while Amazon's Simple Storage Service (S3) calls it an *object*, Rackspace has *cloud files*, and the list goes on. In this chapter, a blob is just a file, and blob storage is a cloud platform service that allows you to store files in the cloud.

A *storage access key* is cryptographically generated by the cloud platform for each storage account, and applications are expected to use this key to access blob storage. It is not easy to safely store this key on client devices, whether for a desktop, mobile, or web application. A compromised key exposes the entire storage account. This pattern will not cover any techniques for protecting keys on client devices, but will instead focus on two other approaches: *public access* and *temporary access*.

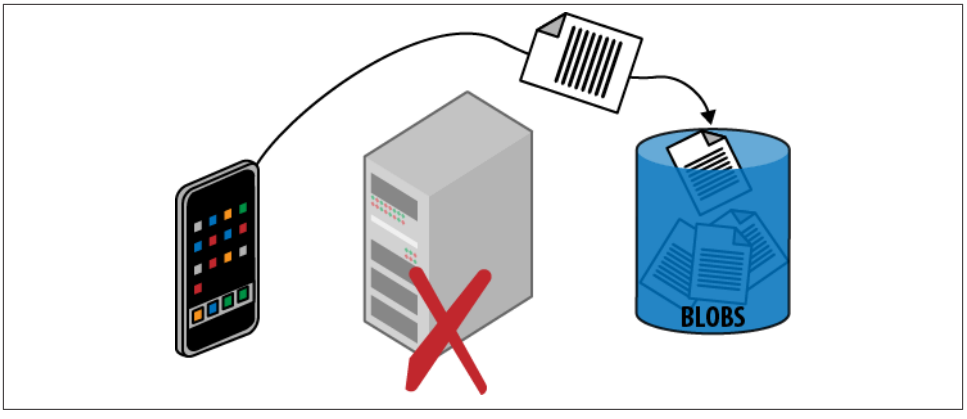


Figure 13-1. Data is uploaded efficiently, going directly from the client to cloud storage without passing through the web tier. The issuing of a temporary access URL—the valet key—is not shown, but is needed before the data can be uploaded.

Public Access

There are two sides to this pattern from the point of view of the client: reading blob data and writing blob data. For blob data that is intended to be publicly visible anyway, blob storage allows us to configure blobs for public read access. Because each blob resource is addressable as a URL, this opens up a considerable number of possibilities. Sharing is easy: just distribute the individual links. Blob URL references can be referenced just like other files from HTML pages so that web applications can benefit from the scalability features of blob storage, reducing the burden on the web tier that would otherwise be responsible for delivering files such as images, videos, JavaScript, and documents.

Anonymous public read access is very powerful. However, anonymous public write access is not supported. Temporary access for both read and write are possible if managed by the application.

Granting Temporary Access

Public read access allows any client knowing the URL to access the blob resource at any time without limit. This may not be what you want. If more control is desired, it is possible to grant temporary access to specific resources by constructing a *temporary access URL* using the storage key. The temporary access URL can be shared with clients who can then access the specific resource, but not all resources, just like with a valet key.

The Valet Key

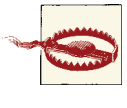
Some cars come with two types of keys: regular keys that work on every lock in the car, and valet keys that provide only limited access. The valet key is useful when you wish to temporarily allow some access, but do not want to give full access. The classic case is valet parking. With valet parking, you turn over your car to a stranger who will park it for you. In exchange for this convenience, you need to give this stranger a key to your car. The valet key is ideal for this, as it will only allow the driver to unlock the doors and start the car; it will not allow access to storage areas. In some newer cars, it will even limit the speed at which the car can be driven.

In the cloud storage world, the temporary access URL serves as a flexible and powerful valet key, allowing efficient and convenient access to storage while still being able to limit that access to only specific resources and during specific times.

Temporary access URLs are not limited to reading, but can be extended to allow other permissions, including writing. Thus is it possible to create one that would enable a client to upload a file directly into a predetermined location in blob storage.

The temporary access URLs are time-limited by your application-supplied expiration. The expiration time might be selected to be just long enough to safely finish an upload, watch a movie, or set to match the expected duration of the login session; this is a policy decision for your application.

Temporary access URLs need to be generated on the server and made available to the client. Reasonable approaches will depend on your application, but might include providing them during login, or separately issuing them on demand to authenticated callers via a web service.



When uploading blobs, transient failures may require retries. See *Busy Signal Pattern* (Chapter 9).

Security Considerations

Any code (in the cloud or elsewhere) with access to the storage access key will be able to create temporary access URLs.

Temporary access URLs are secured through *hashing*, a proven cryptographic technique that requires accesses to the storage key and uses it to create a unique signature for any string, in this case the temporary access URL. The associated hash is checked every time a client attempts to use the temporary access URL; without a correct one, access is denied. Adversaries without access to the storage key cannot tamper with an existing temporary access URL, create a new one, or guess a valid one.

However, as with a real-life valet key, any client in possession of a temporary access URL can use it. Any permissions granted by the temporary access URL are available. Like any other security access token, follow the principle of least privilege and provide only those rights necessary, and only for as long as necessary. Further, when transporting a temporary access URL, do so over a secure channel. Since the cloud platforms support secure HTTPS access to blob storage, clients outside the cloud provider's data center can safely use temporary access URLs. Temporary access URLs can even be used safely to serve pages to a regular web browser; as far as the browser is concerned, it is just requesting resources via HTTPS since the security key is part of the URL (the query string), HTTPS protects the query string during transport, and the cloud storage service handles authorization.

In the physical world, a lost valet key cannot be revoked; the only way to render it useless is by changing the locks. In the cloud, we have other tools such as expiration dates. The Windows Azure cloud platform goes further, supporting temporary access URLs that do not directly contain permissions or an expiration date, but rather reference a policy, known as a stored access policy, which dictates permissions and expiration. This policy can be changed independently of the URLs that reference it. This allows temporary access URLs to be issued that can later be revoked, extended, or have their permissions tweaked. Many URLs can reference the same policy.

There is no limit on the number of temporary access URLs that can be issued and all are independent of one another.

Example: Building PoP on Windows Azure

The Page of Photos (PoP) application (which was described in the Preface) supports publishing photos directly from a smart phone. Since smart phones can now routinely take high-resolution photos and capture HD video, the file may be large (multiple megabytes for a photo to tens of megabytes for a video), making this an especially good use case for this pattern.

From the point of view of the mobile application, it does not care whether it is uploading directly to cloud storage or uploading through a wrapper web service in the web tier; it is the same amount of work. To write a blob into Windows Azure, a mobile application only requires a few lines of code if it is using one of the mobile libraries provided as part of the platform. At the time of this writing, mobile client libraries were available for Android, iOS (iPhone, iPad), and Windows Phone.

Windows Azure Blobs offer a couple of handy features that allow us to streamline this process: public read access and Shared Access Signatures.

Public Read Access

Enabling public read access to blobs is trivial.

Blob storage containers (which are like directories or folders) can be marked as available for either public or private access. Photos stored in PoP are intended to be publicly viewable by anyone, so we can go ahead and mark them all as publicly visible. Once marked, anyone who knows the URL can read it. This is ideal for PoP for uploaded photos and videos, generated thumbnails, and any other size or format variants.

It is not possible to configure blobs in Windows Azure Storage to allow anonymous public updates of any kind; updates always require security credentials (which are described next).

Public read access to photos in PoP will likely result in users sharing URLs to specific photos. This exposes the underlying URL to users. By default, the URL points to a Windows Azure domain, but in our case we would like it to point to a *http://pageofphotos.com* domain; we do this with a vanity domain.

Using a Vanity Domain

Example of a regular photo URL from blob storage:

http://pageofphotos.blob.core.windows.net/photos/daniel.png

Example of a photo URL form blob storage after configuring our vanity domain:

http://www.pageofphotos.com/photos/daniel.png

Shared Access Signatures

A *Shared Access Signature (SAS)* is the Windows Azure Storage feature used to construct temporary access URLs for blobs that have temporary permission for reading or writing blobs.

PoP takes advantage of SAS for blob storage. Windows Azure Storage also features SAS support for its NoSQL database, Windows Azure Tables, as well as Windows Azure Queues.

For PoP, the goal is to permit users to upload photos from a mobile application directly into blob storage without allowing them to upload photos to other user accounts. Permissions are part of the special URL and will not allow one user to interfere with blobs belonging to another user. This level of security is sufficient for PoP. Once the temporary access URL is constructed and delivered to the mobile application, it is a simple matter for the mobile application to upload directly from the mobile device to blob storage, perhaps using one of the mobile client libraries mentioned previously.

SAS for Selective Read Access

The SAS technique can also be used to provide temporary access URLs for reading non-public blob resources. PoP does not require this since all images are public anyway.

If PoP did require that some photos be publicly accessible while others were not, it could use a private container to hold photos and use the SAS technique to provide temporary access to all URLs as needed. Alternatively, private photos could be moved into a separate blob container, allowing public photos in the original blob container to maintain open permissions.

Once a temporary access URL is generated, it is up to your application to safely deliver it to the appropriate client. In the case of the PoP mobile app, the SAS will be returned to the mobile app in response to an authenticated web service call over a secure (HTTPS) connection.

Windows Azure Storage also supports the notion of an *access policy* which is a named collection of permissions. An access policy can be used when creating a SAS. If a SAS

is constructed using an access policy, it is possible to later change that access policy. Any valid SAS will be immediately adjusted to use the new permissions. One common use of access policies is to maintain the ability to revoke permissions if it later becomes necessary.

Related Chapters

- *Eventual Consistency Primer* (Chapter 5)
- *Busy Signal Pattern* (Chapter 9)
- *Network Latency Primer* (Chapter 11)
- *Colocate Pattern* (Chapter 12)
- *CDN Pattern* (Chapter 14)
- *Multisite Deployment Pattern* (Chapter 15)

Summary

Use of this pattern should be considered anywhere it can be safely applied. The ability to manage temporary access for reading and writing makes this a broadly usable pattern. The most common troublesome use case will be an upload directly from a web browser, but reading is well supported, and writing from more flexible clients such as mobile apps is also well supported.

When used with storage containers that support this pattern, applications can avoid having a web page or web service act as a security proxy to read or write data stored in a secure container. This reduces load on the web tier because it is not acting as a middleman for data transfer. Also, the code for implementing all the variants of data passing through is replaced by the far simpler generation and issuing of temporary access URLs, while the upload code is offloaded to the client. The client code should utilize existing helper libraries where available to minimize complexity.

The end result is that applications scale better and the user experience is improved.

CHAPTER 14
CDN Pattern

This pattern focuses on reducing network latency for commonly accessed files through globally distributed edge caching.

The goal is to speed up delivery of application content to users. Content is anything that can be stored in a file such as images, videos, and documents. The Content Delivery Network (CDN) is a service that functions as a globally distributed cache. The CDN keeps copies of application files in many places around the world. When these places are close to users, content does not need to travel as far to be delivered, so it will arrive faster, improving the user experience.

CDN nodes are strategically located around the globe, hopefully close to application users. The CDN has its own URLs that are resolved by a geographic load balancer that directs users to the nearest node, regardless of where the user is.

The flow of files is shown in Figure 14-1. When a CDN URL is requested for the first time, the CDN will retrieve the file from the main source (usually known as the origin server), and then return that to the requesting user while also caching a local copy to satisfy subsequent requests for that file. When a CDN URL is requested and the file has already been cached, the CDN returns a locally cached copy of that file to the requesting user. This is faster than if the user retrieved it from the origin server, which is (presumably) further away. When many users access the same content from a CDN node, the initial request is slower (so the first user to request a file has to wait longer), but subsequent requests are much faster.

A CDN is only helpful for files that will be accessed multiple times. Files that are intended to be rarely accessed or that are for only a single user are usually not good candidates for a CDN.

Each CDN node operates independently of the others.

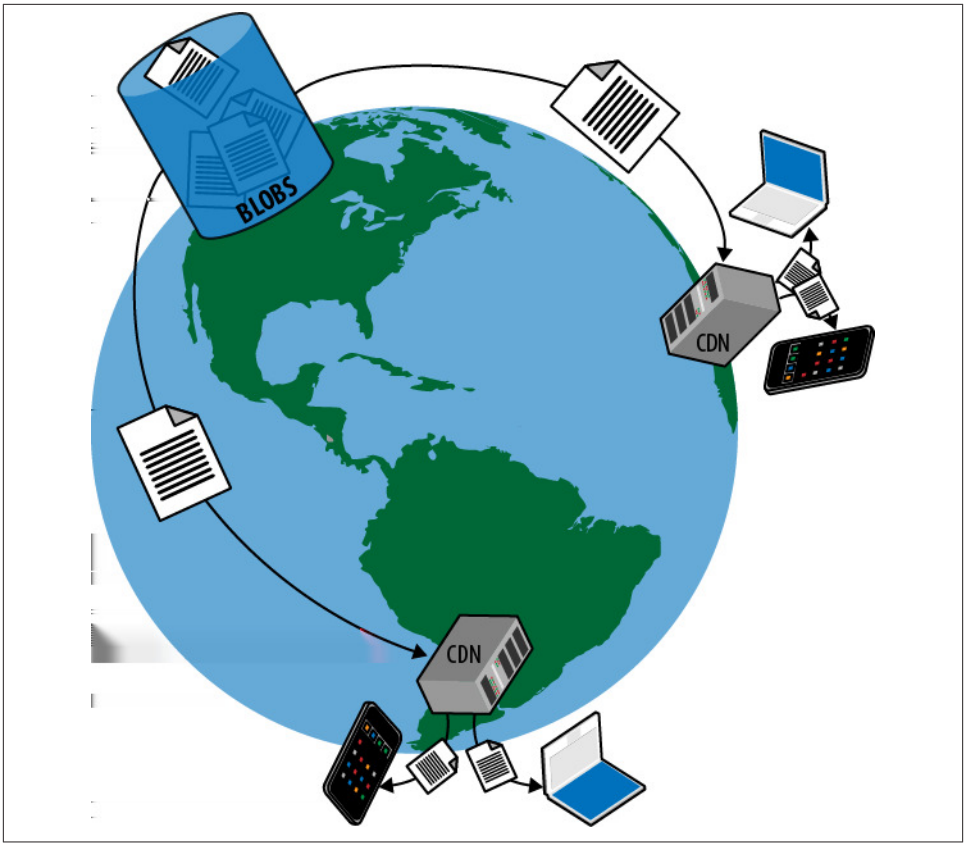


Figure 14-1. Users get content directly from the nearest CDN node. The CDN node fetches content from the source as needed.

Context

The CDN Pattern is effective in dealing with the following challenges:

- Application data is accessed from geographic locations that may not be close to the data center from which it originates
- Multiple clients access the same application data objects (such as HTML, JavaScript, image, video, or other files)
- Application includes large downloads, streaming video, or other heavyweight content delivery

A CDN can effectively reduce the load on other types of nodes that might be serving up content.

Cloud Significance

Cloud platform CDN services are easy to enable and are integrated into the cloud vendor's web-hosted management tool, appear on the same bill, and are supported by the same organization as other services offered within the cloud platform. Some other services, such as blob storage, can be easily CDN-enabled with a few mouse clicks. For these reasons, cloud CDN services can be more convenient than working directly with an independent CDN service provider.

Impact

Scalability, User Experience

Mechanics

Generally speaking, transmitting data over the Internet is faster when the source data and recipient are closer together. One way to bring source data and recipients closer together is by caching copies of the source data in locations that are closer to the recipients. When a user needs the data, retrieving it from the closest location will be faster than retrieving it from the origin, which might potentially be much further away. This practice is commonly known as *edge caching* and is the role of the CDN. Specifically, we will focus on accessing files over HTTP.

To enable the CDN to cache our files, we need to make one change: instead of using our normal domain name in our file URLs, we use one designated for us by the CDN provider. All file resources that we want to be cached by the CDN are changed to the new scheme.

Whenever a user attempts to access a file being managed by the CDN, the request goes to the CDN service to be fulfilled. However, because one of the benefits of the CDN service is that it has cache nodes in many geographical locations, somehow the request needs to be directed to the location nearest to the requesting user. This is usually handled through the *anycast* routing protocol that helps identify the closest CDN node, and the request is smartly routed accordingly.

Even though all users see the same URL for the file, different users will be routed to different CDN nodes. This is the whole point of the CDN: route requests to the nearest CDN node to improve responsiveness.

A CDN is a specialized cache, and as such, is not necessarily already populated with every desired file. This would be the case if the file had never been requested before through this CDN node, or the file had been requested but subsequently expired from the CDN cache. Any resource stored in a CDN has a defined expiration time that your

application sets, so this is something you will want to think about. Some resources, such as a company logo, may be stable for months or years, while other resources such as a user profile photo may have a shorter shelf life of perhaps one hour (users like to update them frequently). The expiration is expressed through the HTTP *Cache-Control* header. Because the Cache-Control caching directive is a long-established Internet standard, web browsers understand and handle this directive with ease. Beyond individual browsers, sometimes proxy servers or other intermediaries will cache, extending the caching benefits beyond a single user and beyond the CDN. Once the resource has expired, the CDN will remove it from its cache. (There are other possible behaviors with some CDN services, such as having the CDN check with the origin server to ascertain whether expired content has indeed changed, and still using the cached object if it has not.)

If a requested file is not available at the CDN node, the CDN service effectively acts as a middleman, and behind the scenes retrieves the file from the origin server, caches (stores) it in the CDN node, and then responds to the original request (which it can now do successfully).

Limitations of CDN

A CDN is not effective for use on resources that are infrequently accessed. A CDN is only efficient for use on resources that are usually accessed at least twice before expiring from the CDN cache.

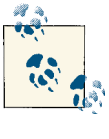
A CDN is not effective for use on resources that change constantly, such as those changing with each request.

A CDN may be a poor choice for content that is not intended for public viewing.

A CDN really shines when used on frequently accessed files that seldom change.

Caches Can Be Inconsistent

The CDN stores copies of resources with a specified expiration date; thus a cached image file might be declared to be valid for one hour, one month, or whatever is appropriate for that resource. Any resource cached in a CDN is potentially stale because there is always a delay between updates to the source copy and propagation to the copy that is cached at the CDN. This is not necessarily a problem, but is a factor to consider carefully when deciding what to cache and for how long.



Caching resources in a CDN is an application of *eventual consistency*: immediate consistency is traded off for performance and scalability benefits.

Example: Building PoP on Windows Azure

The Page of Photos (PoP) application (which was described in the Preface) stores all photos in Windows Azure Blob Storage. To improve the download experience for PoP users, all photo downloads are CDN-enabled.

In Windows Azure Blob Storage, individual blobs are stored in a blob *container* which, among other benefits, establishes a default security context for the blobs within it. Only blobs stored in public containers—where *public* means anyone knowing the URL to a file within the container can view that file—are eligible for caching in the CDN.

Configuring the CDN

Enabling the CDN for blobs is a trivial configuration change at the storage account level. In order for the CDN to sit between users and blob storage, a new domain name is created. This new domain name is system generated. That is, rather than using our own domain name (such as example.com), we use one assigned to us by the CDN service (such as jaromijo1213.vo.msecnd.net). A URL that used to be *http://example.com/maura.png* before the CDN becomes *http://jaromijo.vo.msecnd.net/maura.png* with the CDN. You can optionally configure a vanity domain name of your own to make the CDN addresses appear less chaotic; this vanity domain name is also known as *canonical name* (or CNAME) in DNS terms. PoP takes advantage of this vanity feature, creating *http://cdn.pageofphotos.com* as the CDN domain so that photo URLs will be more consistent with the URL for the main application.

As of this writing, there are 8 Windows Azure data centers worldwide, but 24 Windows Azure CDN node locations. Since there are so many additional geographic locations brought into the mix, use of the CDN network greatly increases global coverage for lower latency content distribution. A resource needs to have been requested at least once in order to be loaded into the CDN cache. The first request for that resource will therefore have a poorer user experience than subsequent requestors. This scenario plays out at each CDN node, as each node needs to fill its own local cache. The scenario also repeats any time a resource is requested, but has expired from the CDN cache.

Each of the 24 CDN nodes operates independently, so each populates independently.

PoP photos do not change frequently, so when the photos are saved in blob storage, a Cache-Control header is set as a property on the individual blob, with the duration set to one month. If we ever needed to update a photo sooner, we would need to change the filename.

The Windows Azure CDN does not currently have built-in support for pre-fetching an object to warm up a particular CDN node. Further, once an object is cached by the CDN, you can't easily change your mind about it since there is no way to evict a particular object from the CDN before its specified cache expiration. Thus, tricks like file renaming or appending a "cache-buster" like a query string are used. For example, we could append a query string so that the cached image *kd1hn.png* becomes *kd1hn.png?v=1* to cause it to reload without finding the one in the current CDN. This requires a change to the reference used to access the image (such as the HTML page referencing it). These are cache tricks that happen to work with the CDN (since it is also a cache honoring HTTP headers); nothing is specific to the Windows Azure Cache.

Cost Considerations

The Windows Azure CDN access charges are the same as directly from blob storage, with no additional at-rest storage charge. Other than the cost of the one additional request needed to populate the CDN node, the cost will be identical to direct-from-blob access. Note that this "one additional request" happens for (a) each resource, (b) each time it is loaded from blob storage (the first time as well as the next time after cache expiration), and (c) in each CDN node from which the resource is accessed.

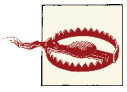
Security Considerations

The Windows Azure CDN only supports caching of files that are already freely visible to the public, so they do not decrease security or increase the attack surface.

The Windows Azure CDN supports access using HTTP or HTTPS. This is primarily a user experience benefit (because the files are already freely visible to the public) and comes into play when a user loads a page using HTTPS. Modern browsers typically display a warning if an HTTPS page references an image as HTTP, complaining about mixed security models. Such images can instead be accessed as HTTPS to improve the user experience.

Additional Capabilities

Though not explored in this chapter, there are other Windows Azure services related to CDN. For example, the CDN can serve files directly from a Web Role. Also, Windows Azure Media Services provide video streaming to the Windows Azure CDN. There are also other integrated services that help applications prepare media for download by many types of devices, such as transcoding services that make content equally accessible across mobile phone brands and desktop operating systems.



As of this writing, the Windows Azure Media Services are currently in preview and not yet supported for production use.

Related Chapters

- *Eventual Consistency Primer* (Chapter 5)
- *Network Latency Primer* (Chapter 11)
- *Colocate Pattern* (Chapter 12)
- *Valet Key Pattern* (Chapter 13)
- *Multisite Deployment Pattern* (Chapter 15)

Summary

Adding CDN support to a cloud application is a great example of a low-friction adoption of a cloud service. Enabling a CDN can be accomplished either programmatically or through a one-time manual configuration via the cloud vendor's web-hosted management tool. This is substantially easier to get started with than traditional CDNs due to the degree of convenience and integration.

Once enabled, this is a great technique for reducing the load on web servers, distributing load across many servers (there are many more CDN locations than data centers), while decreasing network latency. All this helps to both improve scalability and improve user experience.

CHAPTER 15

Multisite Deployment Pattern

This advanced pattern focuses on deploying a single application to more than one data center.

Deploying to multiple data centers helps reduce network latency by routing a client to the nearest data center, which improves the user experience. This also provides the seeds of a solution that can handle failover across data centers and improve availability.

If multisite deployment does not improve user experience and your application does not need cross-data center failover, use of this pattern may be overkill.

Context

The Multisite Deployment Pattern is effective in dealing with the following challenges:

- Users are not clustered near any single data center, but form clusters around multiple data centers or are widely distributed geographically
- Regulations limit options for storing data in specific data centers
- Circumstances require that the public cloud be used in concert with on-premises resources
- Application must be resilient to the loss of a single data center

This pattern helps deal with a user base that is not conveniently clustered in a single geographic area. Some of the reasons for this include use of the application from unpredictable locations during travel, globally distributed mobile applications, and companies with offices distributed across many geographical locations.

This pattern is similar in some ways to *CDN Pattern* (Chapter 14), in that we strive to bring our application closer to our users. The CDN focus is on bringing files closer to our users, and it only helps when sending data to our users, not users sending data to

the application. This pattern is more powerful than a CDN because it brings more application facilities closer to users with lower latency, even when users send data to the application. This pattern is more limited in one way: there are many more CDN nodes than there are full-blown data centers. For these reasons, these two patterns are often used together.

Cloud Significance

Networking cloud services are available for geographic load balancing and cross-data center failover. Data-oriented cloud services are also available for database synchronization and geo-replication of cloud storage. These services, plus access to multiple geographically distributed data centers, simplify some of the most complex aspects of multisite deployment.

Impact

Availability, Reliability, Scalability, User Experience

Mechanics

If all global traffic for an application is served from a single data center, then responsiveness is better near that data center than from more distant regions of the world. Generally speaking, the farther away, the poorer the responsiveness. Which data center to choose then? This pattern is about choosing more than one data center to offer the best available user experience throughout the world.

The major public cloud platforms have multiple data centers on multiple continents. With Windows Azure and Amazon Web Services, you select the specific data center to which you will deploy. Let's assume you choose one data center location in Asia, one in Europe, and one in the United States, and deploy an instance of your application to each data center.

Once your application is deployed to multiple data centers, you need to decide how users will be directed to the appropriate one. In this pattern, we will strive to make this process transparent to users in order to provide the best user experience. In the most basic scenarios, this requires smart routing and data replication services.

To route users to the closest data center, use your cloud platform's service for geographic load balancing and configure it to direct users to the closest data center. These services include the Windows Azure Traffic Manager and Elastic Load Balancing from Amazon Web Services.

You also need to replicate data across all data centers if you want users to see the same data, regardless of the data center to which they route. This is further explored in the example given later in this chapter. The end result can be seen in Figure 15-1, where

data centers replicate data as needed. Since each data center has a local copy of any needed data, users can be conveniently routed to the one nearest to them for the best performance. As we shall see later, this topology also can be leveraged to handle rare scenarios when one of the data centers is not available.

Another possibility is to assign each user to a single data center, though that scenario is not explored.

With geographic load balancing and data replication in place, you have the basics of a multisite deployment. Users will experience better performance than if they always had to directly access data and services from a data center located on another continent.

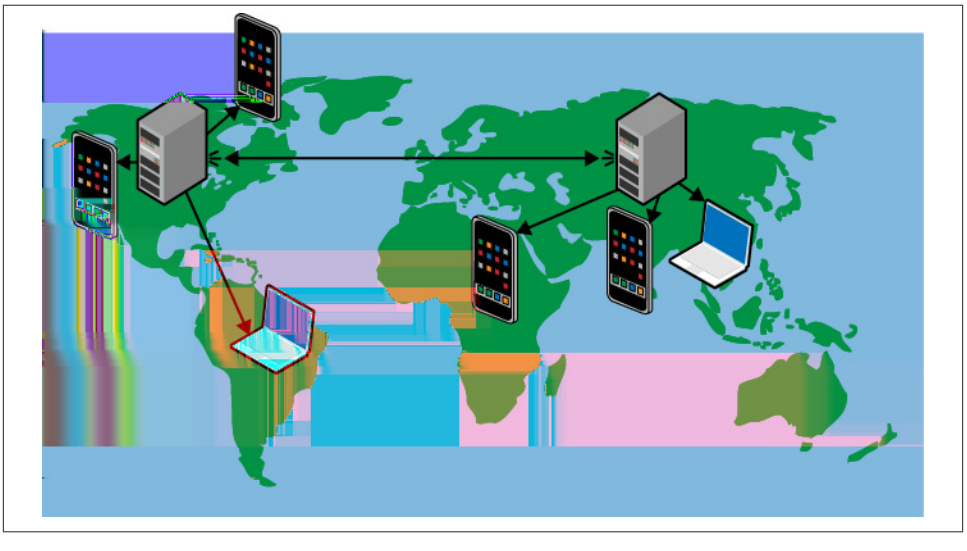


Figure 15-1. Data centers stay in sync so that users can access the closest one, or an alternate in case of failure. The connection spanning the continents represents data center-to-data center synchronization. External user devices are shown accessing the closest data center.

Non-Technical Considerations in Data Center Selection

Sometimes there are non-technical considerations that compel you to separate application tiers across data centers or to choose one data center over another. Sometimes a data center location is chosen in order to comply with government regulations, industry requirements, and other so-called *data sovereignty* issues. For example, some countries

in the European Union (EU) require that applications operating in the EU store personally identifying information within the EU. As another example, the credit card industry has specific expectations of applications handling credit card data. These expectations may further limit or influence options in data center selection.

Finally, consider a business constraint in which a customer is not ready to move a database from its on-premises location into the cloud but wants to access it from cloud servers. Cloud platform services can extend a company's internal network into the cloud with a Virtual Private Network (VPN). It securely handles this scenario so that applications running on cloud resources can easily query databases located elsewhere. This is yet another factor influencing your data center footprint.

Thus, business considerations may override purely performance-based data center selection criteria. While these considerations can impact cloud architecture, a treatment of this complex topic is not the focus of this pattern.

Cost Implications

As of this writing, both Windows Azure and Amazon platforms charge a fee for data leaving the data center, but not for data coming into the data center. Thus, traffic across data centers would incur these charges. Of course, this applies even within a single data center location because remotely accessing a cloud application from outside the data center results in data leaving the data center. In the context of this pattern, this also applies to keeping databases in sync across data centers and accessing on-premises databases.

Deploying your application across multiple data centers has cost implications. Such costs should be weighed against the user experience and the robustness limitations of a single data center. And of course, if multisite deployment is needed, consider the cost and complexity of doing this without the convenience of cloud services.

Failover Across Data Centers

Failover is a feature of an application that enables the application to continue to function using secondary resources when there is a failure with the primary resources. In the context of this pattern, we are concerned with the loss of a data center and the ability of our application to continue to function using the other available data centers. We may want to failover from one data center to another due to a natural disaster, such as a hurricane, or due to a failure by the cloud provider, such as a software bug or configuration error that impacts services.

The tools and techniques used to create a multisite deployment can also help to make your application more failure-resistant. The same services used for geographic load

balancing that route users to the closest data center can be used to account for failover scenarios. In particular, they can be configured to monitor the health of the services to which they are directing users, and if any service is unavailable, traffic will be routed to a healthy instance.

It takes time for the geographic load balancer to figure out that a service is not responding before it fails over to a healthy copy. During that time, application access may result in errors. Client-side retries, such as *Busy Signal Pattern* (Chapter 9), can help shield users from these errors. Once failover is complete, users will be routed to data centers; if these data centers are on other continents, their experience will be degraded, but it will work. This is usually better than not working at all, although that depends on the specific application.

As unhealthy services become healthy again, traffic can be delivered, returning system responsiveness to maximum levels.

This scheme does not guarantee instant or seamless failover. There will be downtime.

One approach to failover is to have the secondary resources already deployed, just in case they are needed. This arrangement is known as either *active/passive* or *active/active*, depending on whether the secondary resources are just sitting around in case they are needed or are in active use (the primary data center accounts for the first "active" resource). Another option is to not have any spare resources that are already running, but to deploy those resources as needed for a failover. This obviously would take longer, but is also less costly. This topic is explored further in the *Example*.

This is part of a larger *disaster recovery (DR)* plan. Many of these concerns parallel those for non-cloud applications, but are beyond the scope of this chapter.

Example: Building PoP on Windows Azure

The Page of Photos (PoP) application (which was described in the Preface) faces a few challenges in providing a seamless user experience across the globe:

- Routing users to the nearest data center so that they have the best user experience
- Replicating account data so that it is available at all data centers, both for account owners and visitors
- Replicating identity information so that we can correctly authenticate a user in any data center and know they are the same user
- Serving photos globally and efficiently

Choosing a Data Center

As of this writing, Windows Azure offers eight geographically distributed data centers:

- Two in Asia: East (Hong Kong) and Southeast (Singapore)
- Two in Europe: North (Netherlands) and West (Ireland)
- Four in the United States: North Central (Illinois), East (Virginia), South Central (Texas), and West (California)

Since PoP users are all over the world, we will deploy to one data center in Asia (Singapore), one in Europe (Ireland), and one in North America (Virginia). However, PoP users should not need to know about the three data centers; using PoP should just work, regardless of where the user is. In other words, *http://www.pageofphotos.com* should be the only address a user ever needs to remember.

Routing to the Closest Data Center

Windows Azure Traffic Manager is a scalable, highly available, easily configured, geographic load balancing service that continually monitors the responsiveness of all application deployments behind the scenes. It ensures that a visitor to *http://www.pageofphotos.com* is transparently directed to the data center that will give the visitor the best response.

Configuring Traffic Manager

Traffic Manager configuration includes specifying a public domain name (*http://www.pageofphotos.com*) that users see, listing the individual instances (*http://pop-asia.cloudapp.net*, *http://pop-eur.cloudapp.net*, *http://pop-usa.cloudapp.net*), and configuring the routing rules for the closest instance. The endpoint names listed are just examples, but show use of a consistent naming pattern that can help avoid confusion and errors.

In our case, as an example, visiting *http://www.pageofphotos.com/timothy* from Asia would be satisfied by the *http://pop-asia.cloudapp.net* instance.

Replicating User Data for Performance

Consider a PoP user in Ireland who posts some new photos. They can be neatly and efficiently stored in the Ireland data center with a minimum of network latency. The collection can be shared with friends in Ireland who also access them from the Ireland data center. So far, so good.

Now consider that a link to this page of photos is emailed to some friends in Boston. Traffic from Boston will be routed to the Virginia data center. What will happen? To make it work, the Virginia data center needs access to the same data that was saved to the Ireland data center. Our approach with PoP is split into two complementary schemes: one for uploaded photos, another for the rest of the account data.

The photos are uploaded to blobs in Window Azure Storage in the nearest data center. The application makes no effort to explicitly replicate these photos to other data centers. Thumbnails for these photos are generated in the same data center [using *Queue-Centric Workflow Pattern* (Chapter 3)] and also stored as blobs. Photos are not replicated to all data centers as we plan to rely on *CDN Pattern* (Chapter 14) to deliver them most efficiently to users.

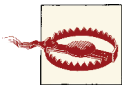
In addition to the photos, there is metadata around the photos such as:

- The account responsible for uploading the photo
- The URL needed for accessing the photo
- The text description of the photo
- And so on.

This data is stored in a Windows Azure SQL Database. The new photo is uploaded to the nearest data center. The metadata is saved to SQL Database in the nearest data center. But, unlike the photo blobs, the SQL Database data is replicated across all three data centers using SQL Database Data Sync service.

Configuring Data Sync

SQL Data Sync service configuration requests the provisioning of a Data Sync Server, creates a Sync Group that will specify the database instances to be kept in sync, then adds the database names of the three instances that will be kept in sync to the Sync Group. It selects the desired sync rules, and lets it run. Syncs happen on a schedule; the most aggressive supported frequency is five minutes between synchronization jobs.



As of this writing, the SQL Database Data Sync service is currently in preview. It is not yet supported for production use.

A user from anywhere in the world will be directed to the nearest data center when accessing a page of photos. It returns the HTML page of photos; the individual photos will be served up from one of the 24 global CDN nodes.

Note the eventual consistency of SQL Database instances. Consider our earlier example of a PoP user from Ireland sharing a page of photos with a friend in Boston. There will be a short window of time during which PoP users in Ireland see new photos, but PoP users in the United States (and Asia) do not; this is eventually resolved as the SQL Database Data Sync service catches up.

Replicating Identity Information for Account Owners

While any anonymous user can view any page of photos, all uploaded photos belong to a PoP account holder. An account on PoP is easily created because it does not manage user credentials, but rather relies on *federated authentication*. PoP is configured to allow users to log in using an *identity provider (IdP)* that they are likely already using. Supported IdPs leveraged by PoP include Google, Yahoo!, and Facebook. (Many other identity providers can be supported, but PoP chooses not to make use of them.) The first time a user signs into their IdP, they give their IdP permission to share login information with PoP. (Actually, the IdP shares with the Access Control Service (ACS), not directly with PoP. PoP gets the information from ACS.) For any IdP supported by PoP, the important item we want is a validated email address. Because each IdP trusted by PoP is reputable, we trust it. We then use the email address as a unique account key in our SQL Database instances. This way, we can tie uploaded photos to a specific user account.

Note further that PoP does not know or store the user's password. Only the underlying IdP does. Using cryptographic techniques, the IdP can securely communicate to PoP that users are who they say they are and pass along a cryptographically secure *claim* that provides their email address. This is all PoP needs. The PoP development effort can be focused on features, not infrastructure.

Federated identity, claims, and identity providers are important concepts for building applications that deal with identity in a cloud-native manner. A whole book could be written to explain these important topics. (In fact, Appendix A references such a book.) The key takeaway for this chapter is that some external entity (an IdP) is telling us the current user owns a specific email address (indicated by a claim) and that we can trust that this is accurate.

The Windows Azure Access Control Service (ACS) acts as an intermediary between PoP and each IdP. It turns out that this is a huge simplification for PoP because there is so much variation in how an IdP can behave. It can be fairly complicated if your application needs to interact with the IdP directly. ACS helps applications manage the relationship with one or more identity providers, abstracting away the implementation details and normalizing them so that claims are consistent when they reach your application.

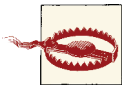
Configuring ACS

Access Control Service configuration consists of establishing a set of rules for each supported provider. The basic process is mostly handled through a set of well-defined steps in a wizard-like user interface; this is appropriate as such configurations are routine. Google and Yahoo! configurations can be completed in the ACS interface, though Facebook requires additional configuration at the Facebook site. ACS also can be configured programmatically. The configuration for PoP is simple: pass through all claims from the IdP (which will include an email address claim), and inject an "Admin" role claim for a few special email accounts (for PoP site administrators).

This description of how ACS helps authentication work for PoP is necessary to provide context for the multisite deployment. The upshot is that this configuration needs to be repeated for each identity provider in each supported data center.

Data Center Failover

Given the measures we have taken to make PoP available in three data centers, what more do we need to do to ensure we can failover in the event one of the data centers fails?



Supporting data center failover is a big decision with many risks and tradeoffs. The approach outlined here is appropriate for the PoP application and business. These risks and tradeoffs may or may not be acceptable for your application and business.

PoP replicates Windows Azure SQL Database instances. As mentioned previously, there is still a window within which data loss could occur. If this is not good enough, it may be challenging to overcome.

PoP has Access Control Service identity providers configured in each data center so that PoP will recognize the data center, regardless of which one is used to authenticate.

PoP uses Windows Azure Storage to save photos and thumbnails as blobs. We have taken no measures to replicate them because the Windows Azure Platform geo-replicates blobs on our behalf. (Windows Azure Storage Tables are also geo-replicated.) Each data center is paired with another on the same continent. In the list of data centers above, the two Asia data centers replicate, the two Europe data centers replicate, and in the United States, North Central replicates with South Central, and East replicates with West. In the extremely rare event of a disaster so great that a data center is lost or unavailable for

an extended period of time, storage will failover to its replication pair. You do not need to do anything. There will be a period of time when blob requests fail, but once the failover is complete, they will resume working correctly. Note, however, that there is still a window within which data loss could occur.

PoP uses the CDN to deliver photos and thumbnails close to the users. Because all of the eight Windows Azure data centers also host CDN nodes, a data center failure could likewise impact the CDN. Furthermore, the other CDN nodes could be impacted independently. The built-in CDN routing logic will notice a failure of a CDN node and change where traffic is routed. There is still a window within which files may not be delivered. Given the nature of the CDN, it is likely that the many distributed CDN nodes will somewhat shield users during the interval where blobs are unavailable and in the process of being failed over because many photos will already be cached around the world.

The final step is to update the configuration in Traffic Manager. Earlier we described how Traffic Manager routes to the closest (most responsive) data center. Traffic Manager also handles failover scenarios. For example, if a hurricane rendered the Virginia data center unavailable, Traffic Manager can route users to Singapore or Ireland (whichever it determines to be faster for any individual user). While the user experience is degraded compared to direct access to the North American data center, it is a better user experience than no availability. There is still a window within which Traffic Manager may route to a failing data center.

Because the data center can failover to resources that are already provisioned and in active use, this is an *active/active* failover configuration.

Considering Disaster Recovery and User Experience

How should use of multisite deployment balance disaster recovery (DR) and user experience (primarily performance) benefits? There is no single right answer, but it is generally helpful to look at DR and user experience as independent concerns that happen to have overlapping solutions. Either could drive the decision to go multisite, or the combined value may be needed to justify a multisite deployment.

Colocation Alternatives

As mentioned earlier, there are a number of non-technical business factors that may limit location options for data storage. In Windows Azure, there are a number of features that might be of use in making the most of such constraints. Briefly, these include:

- Windows Azure Virtual Networking and Windows Azure Connect support a secure Virtual Private Network (VPN) connection between your Windows Azure service and another network. This enables many integration scenarios, such as allowing compute resources running in Windows Azure to access a database running in a private data center.
- Windows Azure Service Bus supports several secure scenarios for communication across firewalls, publish/subscribe to connected devices, and publish/subscribe to disconnected devices. These capabilities enable secure and efficient communication across applications regardless of firewall or data center configurations.
- SQL Data Sync Service use for cloud-to-cloud synchronization was already described. This service can also be used to synchronize between the cloud and on-premises databases.

These services can help integrate with a database or services that are on premises or in another data center.

Related Chapters

- *Horizontally Scaling Compute Pattern* (Chapter 2)
- *Queue-Centric Workflow Pattern* (Chapter 3)
- *Eventual Consistency Primer* (Chapter 5)
- *Node Failure Pattern* (Chapter 10)
- *Network Latency Primer* (Chapter 11)
- *Colocate Pattern* (Chapter 12)
- *Valet Key Pattern* (Chapter 13)
- *CDN Pattern* (Chapter 14)

Summary

Using the Multisite Deployment Pattern primarily helps improve the user experience for a geographically distributed user base. Users need not be all over the world, but at least distributed such that more than one data center provides sufficient value if the goal is to improve performance.

This pattern is also useful for applications requiring a failover strategy in case one data center becomes unavailable. This is a complex subject, but many of the components in this chapter will get you started.

Because the use of this pattern will result in a more complex and more expensive application than a single data center solution, the business value needs to be assessed and compared with the cost.

APPENDIX A

Further Reading

Page of Photos (PoP) Sample

The Page of Photos sample application is used throughout the book. Parts of it have already been implemented using Windows Azure to demonstrate ideas in the book. The code for Page of Photos (minimalist at first, then built out over time by the author and some accomplices) will be shared in a public repo on GitHub.

- Run the Page of Photos (PoP) sample application: *http://www.pageofphotos.com*
- View the Page of Photos (PoP) sample application source code: *http://www.github.com/codingoutloud/pageofphotos*
- Find information about the book and possibly related content in the future: *http://www.cloudarchitecturebook.com*

Resources From Preface and Chapters

- Windows Azure Platform: *http://www.windowsazure.com*
- Amazon Web Services: *http://aws.amazon.com/*
- Google App Engine: *http://developers.google.com/appengine/*
- A NIST Definition of Cloud Computing (SP 800-145 Sept. 2011): *http://csrc.nist.gov/publications/nistpubs/800-145/SP800-145.pdf*
- (NIST) Cloud Computing Synopsis and Recommendations (SP 800-146 May 2012): *http://csrc.nist.gov/publications/nistpubs/800-146/sp800-146.pdf*

Chapter 1

- "A Compuware analysis of 33 major retailers across 10 million home page views showed that a 1-second delay in page load time reduced conversions by 7%." Source: Compuware, April 2011.
- "Google observed that adding a 500-millisecond delay to page response time caused a 20% decrease in traffic" Source: Marissa Mayer, "What Google Knows" talk at Web 2.0 Conf 2006 (11/09/2006): *http://conferences.oreillynet.com/presentations/web2con06/mayer.ppt*
- "Yahoo! observed a 400-millisecond delay caused a 5-9% decrease [in traffic]." And "Amazon.com reported that a 100-millisecond delay caused a 1% decrease in retail revenue." Source: *http://blog.yottaa.com/2010/11/secret-sauce-for-successful-web-site-web-performance-optimization-wpo*
- "Google has started using web-site performance as a signal in its search engine rankings." Source: Using site speed in web search ranking: *http://googlewebmastercentral.blogspot.com/2010/04/using-site-speed-in-web-search-ranking.html*
- Example of a self-inflicted scaling failure: *http://glinden.blogspot.com/2006/11/amazon-crashes-itself-with-promotion.html*
- ITIL: *http://en.wikipedia.org/wiki/Information_Technology_Infrastructure_Library*

Chapter 2

- Windows Azure Storage Analytics: Logs and Metrics: *http://blogs.msdn.com/b/windowsazurestorage/archive/2011/08/03/windows-azure-storage-analytics.aspx*
- Windows Azure Diagnostics: *http://msdn.microsoft.com/en-us/library/windowsazure/gg433048.aspx*
- About Load Balancers: *http://1wt.eu/articles/2006_lb/*

Chapter 3

- Comparing Windows Azure Storage Queues with Amazon Simple Queue Service: *http://gauravmantri.com/2012/04/15/comparing-windows-azure-queue-service-and-amazon-simple-queue-servicesummary/*
- Comparing the two queue services offered by Windows Azure: *http://msdn.microsoft.com/en-us/library/windowsazure/hh767287.aspx*
- Update Message on a Windows Azure Queue: *http://msdn.microsoft.com/en-us/library/windowsazure/hh452234*
- Loose coupling: *http://en.wikipedia.org/wiki/Loose_coupling*

- Long Polling with SignalR for ASP.NET: *http://signalr.net*
- Long Polling with Socket.IO for Node.js: *http://socket.io*
- CQRS Pattern: *http://martinfowler.com/bliki/CQRS.html*
- CQRS: *http://www.cqrsinfo.com/*
- Event Sourcing: *http://martinfowler.com/eaaDev/EventSourcing.html*

Chapter 4

- Enterprise Library 5.0 Integration Pack for Windows Azure (contains WASABi): *http://msdn.microsoft.com/en-us/library/hh680918*
- Hosted services that can be used to monitor and autoscale your Windows Azure applications include AzureWatch from Paraleap Technologies and AzureOps from Opstera.
- Auto-Scaling on Amazon Web Services: *http://aws.amazon.com/autoscaling*
- Ticket Direct case study that sharded databases then consolidated depending on ticket sales: New Zealand-based TicketDirect International: *http://www.microsoft.com/casestudies/Case_Study_Detail.aspx?CaseStudyID=4000005890*

Chapter 5

- Life beyond Distributed Transactions: an Apostate's Opinion (by Pat Helland): *http://www-db.cs.wisc.edu/cidr/cidr2007/papers/cidr07p15.pdf*
- CAP Twelve Years Later: How the "Rules" Have Changed (by Eric Brewer): *http://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed*
- Introducing BASE: *http://queue.acm.org/detail.cfm?id=1394128*
- Introducing Eventual Consistency: *http://queue.acm.org/detail.cfm?id=1466448*
- Eventual consistency in CloudFront: *http://docs.amazonwebservices.com/AmazonCloudFront/latest/DeveloperGuide/Concepts.html*
- Google BigTable: *https://developers.google.com/appengine/docs/python/datastore/overview*
- Amazon Dynamo SOSP paper: *http://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf*
- Azure Storage SOSP paper: *http://sigops.org/sosp/sosp11/current/2011-Cascais/printable/11-calder.pdf*

Chapter 6

- Hadoop: *http://hadoop.apache.org*
- Hadoop on Windows Azure: *http://www.hadooponazure.com*

Chapter 7

- An Unorthodox Approach to Database Design: The Coming of the Shard from High Scalability blog: *http://highscalability.com/unorthodox-approach-database-design-coming-shard*.
- The official source for learning about Federations in Windows Azure SQL Database.
- For anyone interested in Federations for Windows Azure SQL Database, Cihan Biyikoglu's blog is a must-read. Some particularly useful posts are listed below.
  - Implementing MERGE command using SQL Azure Migration Wizard by *@gihuey*: *http://blogs.msdn.com/b/cbiyikoglu/archive/2012/02/20/implementing-alter-federation-merge-at-command-using-sql-azure-migration-wizard-by-gihuey.aspx*.
  - Introduction to Fan-out Queries for Federations in SQL Azure (Part 1): Scalable Queries over Multiple Federation Members, MapReduce Style!: *http://blogs.msdn.com/b/cbiyikoglu/archive/2011/12/29/introduction-to-fan-out-queries-querying-multiple-federation-members-with-federations-in-sql-azure.aspx*.
- Integrated sharding support with Windows Azure SQL Database Federations: *http://blogs.msdn.com/b/cbiyikoglu/archive/2012/02/08/connection-pool-fragmentation-scale-to-100s-of-nodes-with-federations-and-you-won-t-need-to-ever-learn-what-these-nasty-problems-are.aspx*
- Federations: *http://msdn.microsoft.com/en-us/magazine/hh848258.aspx*
- Choosing a shard key in MongoDB: *http://www.mongodb.org/display/DOCS/Choosing+a+Shard+Key*
- SQL Azure Data Sync: *http://msdn.microsoft.com/en-us/library/windowsazure/hh667301.aspx*
- Windows Azure Table Storage service: *http://www.windowsazure.com/en-us/develop/net/how-to-guides/table-services/*
- Generating a GUID as a cluster key with NEWID for Federations on SQL Database: *http://msdn.microsoft.com/en-us/library/ms190348.aspx*

Chapter 8

- Definition of multitenancy: *http://en.wikipedia.org/wiki/Multitenancy*
- Definition of commodity hardware: *http://en.wikipedia.org/wiki/Commodity_hardware*

Chapter 9

- Definition of multitenancy: *http://en.wikipedia.org/wiki/Multitenancy*
- Transient Fault Handling Application Block (Topaz): *http://msdn.microsoft.com/en-us/library/hh680934(v=PandP.50).aspx*
- Scalability Targets for Windows Azure Storage: *http://blogs.msdn.com/b/windowsazurestorage/archive/2010/05/10/windows-azure-storage-abstractions-and-their-scalability-targets.aspx*
- Implementing Retry Logic on Windows Azure: *http://www.davidaiken.com/2011/10/10/implementing-windows-azure-retry-logic/*
- Fault isolation and recovery: *http://www.faqs.org/rfcs/rfc816.html*
- Chaos Monkey from Netflix: *http://techblog.netflix.com/2011/07/netflix-simian-army.html*

Chapter 10

- Understanding Network Failures in Data Centers: Measurement, Analysis, and Implications: *http://research.microsoft.com/en-us/um/people/navendu/papers/greenberg09vl2.pdf*
- How Windows Azure knows a Role Instance (node) is faulty: *http://blogs.msdn.com/b/mcsuksoldev/archive/2010/05/10/how-does-azure-identify-a-faulty-role-instance.aspx*
- Windows Azure Troubleshooting Best Practices: *http://msdn.microsoft.com/en-us/library/windowsazure/hh771389.aspx*
- Updating a Windows Azure deployment, including Fault Domains and Update Domains: *http://msdn.microsoft.com/en-us/library/ff966479.aspx*

Chapter 11

- Ping utility: *http://en.wikipedia.org/wiki/Ping*

- It's the Latency Stupid essay: *http://rescomp.stanford.edu/~cheshire/rants/Latency.html*

Chapter 12

- On the importance of affinity groups: *https://msmvps.com/blogs/nunogodinho/archive/2012/03/04/importance-of-affinity-groups-in-windows-azure.aspx*

Chapter 13

- Windows Azure Toolkits for Mobile Devices (Android, iOS, Windows Phone, and more): *https://github.com/WindowsAzure-Toolkits*
- Restricting Access to Containers and Blobs Windows Azure: *http://msdn.microsoft.com/en-us/library/windowsazure/dd179354*
- Web Browser Same Origin Policy: *http://en.wikipedia.org/wiki/Same_origin_policy*
- Using a Shared Access Signature (REST API): *http://msdn.microsoft.com/en-us/library/windowsazure/ee395415.aspx*
- Rahul Rai's sample code showing access to Windows Azure Blob Storage from HTML 5 Web Browser: *http://code.msdn.microsoft.com/windowsazure/Silverlight-Azure-Blob-3b773e26*
- Trusted Subsystem Design: *http://msdn.microsoft.com/en-us/library/aa905320.aspx*

Chapter 14

- Anycast protocol enables geographic load balancing for CDN: *http://en.wikipedia.org/wiki/Anycast*
- Windows Azure Media Service: *https://www.windowsazure.com/en-us/home/features/media-services/*
- Recorded talk on Windows Azure CDN: *http://channel9.msdn.com/Events/TechEd/NorthAmerica/2011/COS401*

Chapter 15

- Windows Azure SQL Data Sync service: *http://msdn.microsoft.com/en-us/library/windowsazure/hh456371.aspx*

- Windows Azure Traffic Manager: *http://msdn.microsoft.com/en-us/wazplatformtrainingcourse_windowsazuretrafficmanager.aspx*
- A Guide to Claims-Based Identity and Access Control: *http://msdn.microsoft.com/en-us/library/ff423674.aspx*
- Windows Azure Access Control Service: *http://msdn.microsoft.com/en-us/library/windowsazure/gg429786.aspx*
- Automating the Windows Azure Access Control Service (ACS): *http://msdn.microsoft.com/en-us/library/gg185927.aspx*
- ACS automation sample code: *http://acs.codeplex.com/releases/view/57595*
- SQL Azure Point In Time Restore now available in preview: *http://www.microsoft.com/en-us/download/details.aspx?id=28364*
- Business Continuity in Windows Azure SQL Database: *http://msdn.microsoft.com/en-us/library/windowsazure/hh852669.aspx*

Index

Symbols

A

We'd like to hear your suggestions for improving our indexes. Send email to index@oreilly.com.

B

C

D

E

O

P

T

U

V

W

Y

About the Author

Bill Wilder is a hands-on developer, architect, consultant, trainer, speaker, writer, and community leader focused on helping companies and individuals succeed with the cloud using the Windows Azure Platform. Bill began working with Windows Azure when it was unveiled at the Microsoft PDC in 2008 and subsequently founded Boston Azure, the first/oldest Windows Azure user group in the world, in October 2009. Bill is recognized by Microsoft as a Windows Azure MVP and is the author of *Cloud Architecture Patterns*. Bill can be found blogging at *blog.codingoutloud.com* and on Twitter at @codingoutloud.